

XCPC Template Library

Morning Flower
and Evening Oath

Tongji University

双码风算法模板与数学速查

建设稿 · 2026-09-11

状态说明

本稿包含首批已进行本地验证的模块与完整来源覆盖表的入口。尚未完成 kuangbin、WIDA 全部条目的双码风迁移，尚未完成近三年中国赛站逐题覆盖审计，两套最大流已通过洛谷 P3376；其余模块尚无本项目 OJ AC 证据。本地对拍通过不代表对所有输入的绝对正确性证明；适用范围、数值上限与退化约定属于接口的一部分。

目录

使用与验证	v
第一部分 紧凑动态风格	1
第一章 数据结构	2
1.1 并查集	2
1.2 可撤销并查集	2
1.3 树状数组与第 k 小	3
1.4 区间加与区间和	3
1.5 64 位异或线性基	4
第二章 网络流	5
2.1 最大流与方案	5
2.2 势能最小费用最大流	6
2.3 上下界可行循环流	7
第三章 图论	8
3.1 非负权最短路	8
3.2 强连通分量	9
3.3 2-SAT 与任意可行方案	9
3.4 二分图最大匹配	10
3.5 割点与桥	11
第四章 树上算法	12
4.1 重链剖分与 LCA	12
第五章 字符串	13
5.1 KMP、Z、Manacher 与最小表示	13
5.2 AC 自动机出现次数	14
5.3 倍增后缀数组与 LCP	15
5.4 后缀自动机	16
第六章 数论	17
6.1 模运算、素性、CRT 与类欧几里德	17
6.2 Pollard-Rho 因数分解	18
6.3 线性筛、欧拉函数与 Möbius 函数	19
6.4 静态模整数	19
6.5 阶乘组合数	19
第七章 多项式	20
7.1 NTT、形式幂级数、FWT 与递推	20

第八章 代数与数论进阶	22
8.1 模高斯消元、行列式、矩阵树定理	22
8.2 杜教筛	23
8.3 扩展 BSGS	24
第九章 计算几何	25
9.1 整数精确二维几何	25
9.2 浮点点线圆构造	26
第十章 优化与可持久化	28
10.1 Max-plus 矩阵幂	28
10.2 离散李超树与最优直线编号	29
10.3 主席树：静态区间第 k 小	30
第十一章 图论进阶	31
11.1 矩形二分图最小权匹配	31
11.2 有向最小树形图	32
11.3 无向全局最小割	33
第十二章 精确几何进阶	34
12.1 整数最近点对与 Minkowski 和	34
12.2 整数三维共线、共面与方向	35
第二部分 传统接口风格	36
第十三章 数据结构	37
13.1 并查集	37
13.2 可撤销并查集	38
13.3 树状数组与第 k 小	39
13.4 区间加与区间和	40
13.5 64 位异或线性基	41
第十四章 网络流	42
14.1 最大流与方案	42
14.2 势能最小费用最大流	43
14.3 上下界可行循环流	45
第十五章 图论	46
15.1 非负权最短路	46
15.2 强连通分量	46
15.3 2-SAT 与任意可行方案	48
15.4 二分图最大匹配	48
15.5 割点与桥	50
第十六章 树上算法	51
16.1 重链剖分与 LCA	51

第十七章 字符串	53
17.1 KMP、Z、Manacher 与最小表示	53
17.2 AC 自动机出现次数	54
17.3 倍增后缀数组与 LCP	56
17.4 后缀自动机	56
第十八章 数论	58
18.1 模运算、素性、CRT 与类欧几里德	58
18.2 Pollard-Rho 因数分解	60
18.3 线性筛、欧拉函数与 Möbius 函数	61
18.4 静态模整数	62
18.5 阶乘组合数	62
第十九章 多项式	63
19.1 NTT、形式幂级数、FWT 与递推	63
第二十章 代数与数论进阶	67
20.1 模高斯消元、行列式、矩阵树定理	67
20.2 杜教筛	69
20.3 扩展 BSGS	70
第二十一章 计算几何	71
21.1 整数精确二维几何	71
21.2 浮点点线圆构造	73
第二十二章 优化与可持久化	76
22.1 Max-plus 矩阵幂	76
22.2 离散李超树与最优直线编号	76
22.3 主席树：静态区间第 k 小	78
第二十三章 图论进阶	79
23.1 矩形二分图最小权匹配	79
23.2 有向最小树形图	80
23.3 无向全局最小割	81
第二十四章 精确几何进阶	82
24.1 整数最近点对与 Minkowski 和	82
24.2 整数三维共线、共面与方向	84
第三部分 数学速查与维护	85
第二十五章 数学速查：条件先于公式	86
25.1 整除、卷积与筛法	86
25.2 模运算的适用条件	86
25.3 组合计数	86
25.4 群作用计数	87
25.5 生成函数与递推	87
25.6 线性代数与生成树	87
25.7 概率、期望与博弈	87
25.8 数值算法与几何	87

第二十六章 来源与覆盖维护	88
26.1 参考基线	88
26.2 后续覆盖项	88

使用与验证

英文署名已于 QOJ 队伍账号 `ucup-team7502` 核对。两套风格依据训练提交 2932575、2930805、2931279 提取，不擅自将共享队号的提交归属到个人。

代码采用 C++20。动态版默认需要 `bits/stdc++.h` 和 `cassert`；传统版各头文件列明依赖。抄写单个 `struct` 时需同时抄写其依赖结构。同名的不同风格实现通常用于不同提交，不建议混在同一编译单元中。点号除特别注明外为 1-based，字符串和多项式为 0-based；所有区间约定应以条目说明为准。

测试源码位于 `tests/`，固定种子便于复现。验证包含独立暴力参考、边界案例、静态与动态实现、AddressSanitizer 和 UndefinedBehaviorSanitizer。`verification/` 保存实际运行记录与源码摘要；OJ 状态只在获得可查提交结果后登记。最大流记录为 <https://www.luogu.com.cn/record/297501480> 与 <https://www.luogu.com.cn/record/297502364>。

原始来源目录共登记 713 项，见 `docs/coverage.csv`。它包含不同版本、例题及公式，不应将其数目视为独立算法数，也不能将“已登记”视为“已完成”。

第一部分

紧凑动态风格

第一章 数据结构

1.1 并查集

$\text{init}(n)$ 、 $\text{find}(x)$ 、 $\text{merge}(x,y)$ ，点号 $1..n$ ； $\text{siz}[\text{find}(x)]$ 为集合大小。路径压缩与按大小合并，均摊 $O(\alpha(n))$ 。

```
1 struct DSU
2 {
3     vector<int> fa,siz;
4     DSU(int n=0) { init(n); }
5     void init(int n)
6     {
7         fa.resize(n+1); iota(fa.begin(),fa.end(),0);
8         siz.assign(n+1,1);
9     }
10    int find(int x)
11    {
12        while (x!=fa[x]) { fa[x]=fa[fa[x]]; x=fa[x]; }
13        return x;
14    }
15    bool merge(int x,int y)
16    {
17        x=find(x); y=find(y);
18        if (x==y) return false;
19        if (siz[x]<siz[y]) swap(x,y);
20        fa[y]=x; siz[x]+=siz[y]; return true;
21    }
22 };
```

1.2 可撤销并查集

构造后 $\text{snapshot}()$ 保存历史长度， $\text{rollback}(t)$ 回退。禁止路径压缩； find/merge 为 $O(\log n)$ ，回退每次成功合并 $O(1)$ 。

```
1 struct RollbackDSU
2 {
3     vector<int> fa,siz;
4     vector<pair<int,int>> his;
5     RollbackDSU(int n):fa(n+1),siz(n+1,1)
6     { iota(fa.begin(),fa.end(),0); }
7     int find(int x) const
8     { while (x!=fa[x]) x=fa[x]; return x; }
9     int snapshot() const { return (int)his.size(); }
10    bool merge(int x,int y)
11    {
12        x=find(x); y=find(y);
13        if (x==y) return false;
14        if (siz[x]<siz[y]) swap(x,y);
15        his.push_back({y,siz[x]}); fa[y]=x; siz[x]+=siz[y];
16        return true;
17    }
18    void rollback(int t)
19    {
20        assert(0<=t&&t<=snapshot());
21        while (snapshot()>t)
22        {
23            auto [y,s]=his.back(); his.pop_back();
24            siz[fa[y]]=s; fa[y]=y;
25        }
26    }
27 };
```

1.3 树状数组与第 k 小

$\text{add}(x,v)$ 、 $\text{sum}(x)$ 、 $\text{query}(l,r)$ ，下标 $1..n$ ，闭区间； $\text{kth}(k)$ 要求所有频率非负且 $k>0$ ，超过总和返回 $n+1$ 。单次 $O(\log n)$ 。

```

1 template<class T=long long> struct Fenwick
2 {
3     int n; vector<T> a;
4     Fenwick(int n=0):n(n),a(n+1) {}
5     void add(int x,T v)
6     { assert(x>=1); for (;x<=n;x+=x&-x) a[x]+=v; }
7     T sum(int x) const
8     { T ans=0; for (;x;x-=x&-x) ans+=a[x]; return ans; }
9     T query(int l,int r) const { return sum(r)-sum(l-1); }
10    // Nonnegative frequencies; returns n+1 if k exceeds total.
11    int kth(T k) const
12    {
13        assert(k>0); int x=0,b=1;
14        while (b<=n/2) b*=2;
15        for (;b>=1)
16            if (x+b<=n&&a[x+b]<k) { k-=a[x+b]; x+=b; }
17        return x+1;
18    }
19 };

```

1.4 区间加与区间和

初值全零； $\text{add}(l,r,v)$ 、 $\text{query}(l,r)$ ， $1\leq l\leq r\leq n$ 。维护 sum 与加法 tag ；所有中间和与长度乘积须在 long long 范围内。单次 $O(\log n)$ 。

```

1 struct LazySeg
2 {
3     int n; vector<long long> sum,tag;
4     LazySeg(int n):n(n),sum(4*n+4),tag(4*n+4) { assert(n>0); }
5     void apply(int p,int l,int r,long long v)
6     { sum[p]+=(r-l+1)*v; tag[p]+=v; }
7     void push(int p,int l,int r)
8     {
9         int m=(l+r)/2;
10        apply(p*2,l,m,tag[p]); apply(p*2+1,m+1,r,tag[p]);
11        tag[p]=0;
12    }
13    void add(int l,int r,long long v) { add(1,1,n,l,r,v); }
14    void add(int p,int l,int r,int ql,int qr,long long v)
15    {
16        if (ql<=l&&r<=qr) { apply(p,l,r,v); return; }
17        push(p,l,r); int m=(l+r)/2;
18        if (ql<=m) add(p*2,l,m,ql,qr,v);
19        if (qr>m) add(p*2+1,m+1,r,ql,qr,v);
20        sum[p]=sum[p*2]+sum[p*2+1];
21    }
22    long long query(int l,int r) { return query(1,1,n,l,r); }
23    long long query(int p,int l,int r,int ql,int qr)
24    {
25        if (ql<=l&&r<=qr) return sum[p];
26        push(p,l,r); int m=(l+r)/2; long long ans=0;
27        if (ql<=m) ans+=query(p*2,l,m,ql,qr);
28        if (qr>m) ans+=query(p*2+1,m+1,r,ql,qr);
29        return ans;
30    }
31 };

```

1.5 64 位异或线性基

$\text{insert}(x)$ 返回是否增加秩； $\text{query}(x)$ 返回 x 与所张成空间异或后的最大无符号值。单次 $O(64)$ ，不提供第 k 小。

```
1 struct XorBasis
2 {
3     unsigned long long a[64]{};
4     bool insert(unsigned long long x)
5     {
6         for (int i=63;i>=0;i--) if (x>>i&1)
7         {
8             if (!a[i]) { a[i]=x; return true; }
9             x^=a[i];
10        }
11        return false;
12    }
13    unsigned long long query(unsigned long long x=0) const
14    { for (int i=63;i>=0;i--) x=max(x,x^a[i]); return x; }
15 };
```

第二章 网络流

2.1 最大流与方案

Dinic(n), add(u,v,c) 返回偶数原边编号; flow(s,t,limit) 返回本次新增流; used(id) 查询原边净流, cut(s) 返回残量可达点集。允许重边、自环, 容量非负, $s \neq t$; 重复求流须使用相同源汇。只有求尽最大流后 cut 才是最小割。一般 $O(V^2E)$, 递归增广深度 $O(V)$ 。总流不得超过 long long 上限。

```
1 struct Dinic
2 {
3     using ll=long long;
4     struct Edge { int from,to; ll cap,initial; };
5     int n; vector<Edge> e; vector<vector<int>> g;
6     vector<int> dep,cur;
7     Dinic(int n):n(n),g(n+1),dep(n+1),cur(n+1) {}
8     int add(int u,int v,ll c)
9     {
10         assert(c>=0); int id=(int)e.size();
11         e.push_back({u,v,c,c}); e.push_back({v,u,0,0});
12         g[u].push_back(id); g[v].push_back(id+1); return id;
13     }
14     bool bfs(int s,int t)
15     {
16         fill(dep.begin(),dep.end(),-1); queue<int> q;
17         dep[s]=0; q.push(s);
18         while (!q.empty())
19         {
20             int u=q.front(); q.pop();
21             for (int id:g[u]) if (e[id].cap&&dep[e[id].to]<0)
22                 { dep[e[id].to]=dep[u]+1; q.push(e[id].to); }
23         }
24         return dep[t]>=0;
25     }
26     ll dfs(int u,int t,ll f)
27     {
28         if (u==t) return f;
29         for (int &i=cur[u];i<(int)g[u].size();i++)
30         {
31             int id=g[u][i],v=e[id].to;
32             if (!e[id].cap||dep[v]!=dep[u]+1) continue;
33             ll d=dfs(v,t,min(f,e[id].cap));
34             if (d) { e[id].cap-=d; e[id^1].cap+=d; return d; }
35         }
36         return 0;
37     }
38     // Returns additional flow; reuse only with the same source/sink.
39     ll flow(int s,int t,ll limit=LLONG_MAX)
40     {
41         assert(s!=t&&limit>=0); ll ans=0;
42         while (ans<limit&&bfs(s,t))
43         {
44             fill(cur.begin(),cur.end(),0);
45             while (ans<limit)
46             {
47                 ll d=dfs(s,t,limit-ans);
48                 if (!d) break;
49                 ans+=d;
50             }
51         }
52         return ans;
53     }
54     ll used(int id) const { return e[id].initial-e[id].cap; }
55     vector<int> cut(int s)
56     {
```

```

57     bfs(s,s); vector<int> ans;
58     for (int u=1;u<=n;u++) if (dep[u]>=0) ans.push_back(u);
59     return ans;
60 }
61 };

```

2.2 势能最小费用最大流

$\text{add}(u,v,\text{cap},\text{cost})$ 返回边编号, $\text{flow}(s,t,\text{limit})$ 返回新增流量及 128 位费用, $\text{used}(\text{id})$ 返回原边流量。允许负费用, 初始残量网络必须没有负环; 检测到负环抛异常。Bellman-Ford 初始化 $O(VE)$, 每次增广 $O(E \log V)$; 增广次数不能简单假设为 $O(V)$ 。另要求 $\sum(\text{cap} \times |\text{cost}|) < 2^{120}$, 防止总费用超过 int128 。

```

1 struct MinCostFlow
2 {
3     using ll=long long; using i128=__int128_t;
4     struct Edge { int from,to; ll cap,cost,initial; };
5     int n; vector<Edge> e; vector<vector<int>> g;
6     MinCostFlow(int n):n(n),g(n+1) {}
7     int add(int u,int v,ll cap,ll cost)
8     {
9         assert(cap>=0&&cost!=LLONG_MIN); int id=(int)e.size();
10        e.push_back({u,v,cap,cost,cap});
11        e.push_back({v,u,0,-cost,0});
12        g[u].push_back(id); g[v].push_back(id+1); return id;
13    }
14    ll used(int id) const { return e[id].initial-e[id].cap; }
15    // Negative costs allowed; initial residual graph must have no negative cycle.
16    pair<ll,i128> flow(int s,int t,ll limit=LLONG_MAX)
17    {
18        assert(s!=t&&limit>=0);
19        const i128 inf=i128(1)<<120;
20        vector<i128> h(n+1,0),d(n+1);
21        vector<int> pre(n+1);
22        // Super-source Bellman-Ford: feasible potentials for every vertex.
23        for (int it=0;it<n;it++)
24        {
25            bool changed=false;
26            for (auto &a:e) if (a.cap&&h[a.to]>h[a.from]+a.cost)
27            { h[a.to]=h[a.from]+a.cost; changed=true; }
28            if (!changed) break;
29            if (it==n-1) throw invalid_argument("negative cycle");
30        }
31        ll f=0; i128 cost=0;
32        while (f<limit)
33        {
34            fill(d.begin(),d.end(),inf); d[s]=0;
35            priority_queue<pair<i128,int>,vector<pair<i128,int>>,greater<pair<i128,int>>> q;
36            q.push({0,s});
37            while (!q.empty())
38            {
39                auto [du,u]=q.top(); q.pop();
40                if (du!=d[u]) continue;
41                for (int id:g[u])
42                {
43                    auto &a=e[id];
44                    i128 nd=du+a.cost+h[u]-h[a.to];
45                    if (a.cap&&nd<d[a.to])
46                    { d[a.to]=nd; pre[a.to]=id; q.push({nd,a.to}); }
47                }
48            }
49            if (d[t]==inf) break;
50            for (int u=1;u<=n;u++) if (d[u]!=inf) h[u]+=d[u];
51            ll take=limit-f;
52            for (int u=t;u!=s;u=e[pre[u]].from)
53                take=min(take,e[pre[u]].cap);
54            for (int u=t;u!=s;u=e[pre[u]].from)
55            {
56                int id=pre[u]; e[id].cap-=take; e[id^1].cap+=take;

```

```

57         cost+=i128(take)*e[id].cost;
58     }
59     f+=take;
60 }
61 return {f,cost};
62 }
63 };

```

2.3 上下界可行循环流

$\text{add}(u,v,lo,hi)$, $0 \leq lo \leq hi$; $\text{solve}()$ 仅调用一次, 返回可行性, 仅成功后读取 $\text{used}(id)$ 。约束为每点流量守恒; 不是带源汇最大流。所有上下界及其总和须在 long long 范围内。依赖 Dinic。

```

1 struct BoundedCirculation
2 {
3     using ll=long long;
4     int n; Dinic g; vector<ll> balance,lower;
5     vector<int> ids; bool solved=false,feasible=false;
6     BoundedCirculation(int n):n(n),g(n+2),balance(n+1) {}
7     int add(int u,int v,ll lo,ll hi)
8     {
9         assert(!solved&&0<=lo&&lo<=hi);
10        int id=ids.size(); ids.push_back(g.add(u,v,hi-lo)); lower.push_back(lo);
11        balance[u]-=lo; balance[v]+=lo; return id;
12    }
13    bool solve()
14    {
15        assert(!solved); solved=true; ll need=0;
16        for (int u=1;u<=n;u++)
17            if (balance[u]>0) { g.add(n+1,u,balance[u]); need+=balance[u]; }
18            else if (balance[u]<0) g.add(u,n+2,-balance[u]);
19        return feasible=(g.flow(n+1,n+2)==need);
20    }
21    ll used(int id) const
22    { assert(feasible); return lower[id]+g.used(ids[id]); }
23 };

```

第三章 图论

3.1 非负权最短路

$\text{add}(u,v,w)$ 、 $\text{run}(s)$ ， dis 为距离， pre 为前驱， $\text{path}(t)$ 返回点序列。不可达用 LLONG_MAX ；所有可表示的最短距离必须严格小于该哨兵， $w \geq 0$ 。 $O((V+E)\log V)$ 。

```
1 struct Dijkstra
2 {
3     using ll=long long;
4     static constexpr ll inf=LLONG_MAX;
5     int n; vector<vector<pair<int,ll>>> g;
6     vector<ll> dis; vector<int> pre;
7     Dijkstra(int n):n(n),g(n+1) {}
8     void add(int u,int v,ll w) { assert(w>=0); g[u].push_back({v,w}); }
9     void run(int s)
10    {
11        dis.assign(n+1,inf); pre.assign(n+1,-1); dis[s]=0;
12        priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<pair<ll,int>>> q;
13        q.push({0,s});
14        while (!q.empty())
15        {
16            auto [d,u]=q.top(); q.pop();
17            if (d!=dis[u]) continue;
18            for (auto [v,w]:g[u] if (w<inf-d&&d+w<dis[v])
19                { dis[v]=d+w; pre[v]=u; q.push({dis[v],v}); }
20        }
21    }
22    vector<int> path(int t) const
23    {
24        if (dis[t]==inf) return {};
25        vector<int> ans;
26        for (;t!=-1;t=pre[t]) ans.push_back(t);
27        reverse(ans.begin(),ans.end()); return ans;
28    }
29 };
```

3.2 强连通分量

$\text{add}(u,v)$ 、 $\text{run}()$ 、 $\text{bel}[1..n]$ 为分量编号， cnt 为数量；分量编号按缩点图拓扑序递增。迭代 Kosaraju 避免递归爆栈； $O(V+E)$ ，同时存正反图。

```

1 struct SCC
2 {
3     int n,cnt=0; vector<vector<int>> g,rg;
4     vector<int> bel;
5     SCC(int n):n(n),g(n+1),rg(n+1) {}
6     void add(int u,int v) { g[u].push_back(v); rg[v].push_back(u); }
7     // Iterative Kosaraju; component IDs are in topological order, 1..cnt.
8     void run()
9     {
10         vector<int> vis(n+1),order; cnt=0; bel.assign(n+1,0);
11         for (int s=1;s<=n;s++) if (!vis[s])
12         {
13             vector<pair<int,int>> st{{s,0}}; vis[s]=1;
14             while (!st.empty())
15             {
16                 int u=st.back().first,&i=st.back().second;
17                 if (i==(int)g[u].size()) { order.push_back(u); st.pop_back(); }
18                 else
19                 {
20                     int v=g[u][i++];
21                     if (!vis[v]) { vis[v]=1; st.push_back({v,0}); }
22                 }
23             }
24         }
25         reverse(order.begin(),order.end());
26         for (int s:order) if (!bel[s])
27         {
28             ++cnt; vector<int> st{s}; bel[s]=cnt;
29             while (!st.empty())
30             {
31                 int u=st.back(); st.pop_back();
32                 for (int v:rg[u]) if (!bel[v]) { bel[v]=cnt; st.push_back(v); }
33             }
34         }
35     }
36 };

```

3.3 2-SAT 与任意可行方案

$\text{add}(x,a,y,b)$ 添加 $(x=a)$ 或 $(y=b)$ ，变量 $1..n$ ； $\text{solve}()$ 返回可满足性，仅成功时读取 ans 。依赖同文件 SCC； $O(V+E)$ ，不保证字典序最小。

```

1 struct TwoSAT
2 {
3     int n; SCC g; vector<int> ans;
4     TwoSAT(int n):n(n),g(2*n) {}
5     int id(int x,bool value) const { return 2*x-1+value; }
6     void add(int x,bool a,int y,bool b)
7     { g.add(id(x,!a),id(y,b)); g.add(id(y,!b),id(x,a)); }
8     bool solve()
9     {
10         g.run(); ans.assign(n+1,0);
11         for (int x=1;x<=n;x++)
12         {
13             if (g.bel[id(x,0)]==g.bel[id(x,1)]) return false;
14             ans[x]=g.bel[id(x,1)]>g.bel[id(x,0)];
15         }
16         return true;
17     }
18 };

```

3.4 二分图最大匹配

左右两侧分别编号 $1..n$ 、 $1..m$ ； $\text{add}(u,v)$ ， $\text{solve}()$ 返回匹配数， $l[u]$ 与 $r[v]$ 为匹配端点，0 表示未匹配。分层增广；本版本保守按 $O(VE)$ 估计，不宣称已经验证 Hopcroft-Karp 最紧复杂度。

```

1 struct BipartiteMatching
2 {
3     int n,m; vector<vector<int>> g; vector<int> l,r,dep;
4     BipartiteMatching(int n,int m):n(n),m(m),g(n+1),l(n+1),r(m+1),dep(n+1) {}
5     void add(int u,int v) { g[u].push_back(v); }
6     bool bfs()
7     {
8         queue<int> q; fill(dep.begin(),dep.end(),-1);
9         for (int u=1;u<=n;u++) if (!l[u]) { dep[u]=0; q.push(u); }
10        bool ok=false;
11        while (!q.empty())
12        {
13            int u=q.front(); q.pop();
14            for (int v:g[u])
15                if (!r[v]) ok=true;
16                else if (dep[r[v]]<0) { dep[r[v]]=dep[u]+1; q.push(r[v]); }
17        }
18        return ok;
19    }
20    bool dfs(int u)
21    {
22        for (int v:g[u]) if (!r[v]||(dep[r[v]]==dep[u]+1&&dfs(r[v])))
23            { l[u]=v; r[v]=u; return true; }
24        dep[u]=-1; return false;
25    }
26    int solve()
27    {
28        int ans=0;
29        for (int u=1;u<=n;u++) ans+=(l[u]!=0);
30        while (bfs()) for (int u=1;u<=n;u++) if (!l[u]) ans+=dfs(u);
31        return ans;
32    }
33 };

```

3.5 割点与桥

`add(u,v)` 返回无向边编号; `run()` 后 `cut[u]` 标识割点、`bridge[id]` 标识桥。按边编号跳过父边, 支持重边、自环、不连通图。 $O(V+E)$, 递归深度可能 $O(V)$ 。

```

1 struct Lowlink
2 {
3     int n,timer=0; vector<vector<pair<int,int>>> g;
4     vector<int> dfn,low,cut,bridge; int edges=0;
5     Lowlink(int n):n(n),g(n+1) {}
6     int add(int u,int v)
7     { g[u].push_back({v,edges}); g[v].push_back({u,edges}); return edges++; }
8     void dfs(int u,int pe)
9     {
10         dfn[u]=low[u]=++timer; int children=0;
11         for (auto [v,id]:g[u])
12         {
13             if (id==pe) continue;
14             if (!dfn[v])
15             {
16                 ++children; dfs(v,id); low[u]=min(low[u],low[v]);
17                 if (low[v]>dfn[u]) bridge[id]=1;
18                 if (pe!=-1&&low[v]>=dfn[u]) cut[u]=1;
19             }
20             else low[u]=min(low[u],dfn[v]);
21         }
22         if (pe==--1&&children>1) cut[u]=1;
23     }
24     void run()
25     {
26         timer=0; dfn.assign(n+1,0); low=dfn; cut=dfn; bridge.assign(edges,0);
27         for (int u=1;u<=n;u++) if (!dfn[u]) dfs(u,-1);
28     }
29 };

```

第四章 树上算法

4.1 重链剖分与 LCA

`add(u,v)` 建无向树; `build(root)` 后读取 `fa`、`dep`、`siz`、`son`、`top`、`dfn`、`rk`。子树对应 $[dfn[u], dfn[u] + siz[u] - 1]$ 。
`lca(u,v)` 为 $O(\log n)$, `path(u,v,work,edge)` 分解路径; `work(l,r)` 仅适用于可交换聚合, 边权模式排除 LCA 位置。
输入必须是连通无环树, 建树 $O(n)$, 无递归。

```
1 struct HLD
2 {
3     int n,timer=0; vector<vector<int>> g;
4     vector<int> fa,dep,siz,son,top,dfn,rk;
5     HLD(int n):n(n),g(n+1),fa(n+1),dep(n+1),siz(n+1),son(n+1),top(n+1),dfn(n+1),rk(n+1) {}
6     void add(int u,int v) { g[u].push_back(v); g[v].push_back(u); }
7     // Connected tree only; iterative build avoids chain-shaped DFS stack overflow.
8     void build(int root=1)
9     {
10         fill(fa.begin(),fa.end(),0); fill(son.begin(),son.end(),0);
11         vector<int> order{root}; fa[root]=root; dep[root]=0;
12         for (int i=0;i<(int)order.size();i++)
13         {
14             int u=order[i];
15             for (int v:g[u]) if (v!=fa[u])
16                 { fa[v]=u; dep[v]=dep[u]+1; order.push_back(v); }
17         }
18         assert((int)order.size()==n);
19         for (int i=n-1;i>=0;i--)
20         {
21             int u=order[i]; siz[u]=1;
22             for (int v:g[u]) if (fa[v]==u&&v!=u)
23                 { siz[u]+=siz[v]; if (!son[u]||siz[v]>siz[son[u]]) son[u]=v; }
24         }
25         timer=0; vector<pair<int,int>> st{{root,root}};
26         while (!st.empty())
27         {
28             auto [u,t]=st.back(); st.pop_back();
29             for (;u=son[u])
30             {
31                 top[u]=t; dfn[u]=++timer; rk[timer]=u;
32                 for (int v:g[u]) if (fa[v]==u&&v!=u&&v!=son[u]) st.push_back({v,v});
33             }
34         }
35     }
36     int lca(int u,int v) const
37     {
38         while (top[u]!=top[v])
39             { if (dep[top[u]]<dep[top[v]]) swap(u,v); u=fa[top[u]]; }
40         return dep[u]<dep[v]?u:v;
41     }
42     // Commutative operations only; edge=true excludes the LCA's position.
43     template<class F> void path(int u,int v,F work,bool edge=false) const
44     {
45         while (top[u]!=top[v])
46         {
47             if (dep[top[u]]<dep[top[v]]) swap(u,v);
48             work(dfn[top[u]],dfn[u]); u=fa[top[u]];
49         }
50         if (dep[u]>dep[v]) swap(u,v);
51         if (dfn[u]+edge<=dfn[v]) work(dfn[u]+edge,dfn[v]);
52     }
53 };
```

第五章 字符串

5.1 KMP、Z、Manacher 与最小表示

字符串及返回位置均 0-based。prefix(s)、z(s); match(s,t) 查非空模式串并返回重叠匹配起点。manacher 返回奇数半径（含中心）和偶数半径（中心在 i 前）; rotation 返回最小循环表示起点，空串返回 0。各 $O(n+m)$ 。

```
1 struct StringAlgo
2 {
3     static vector<int> prefix(const string &s)
4     {
5         int n=(int)s.size(); vector<int> p(n);
6         for (int i=1;i<n;i++)
7         {
8             int j=p[i-1];
9             while (j&& s[i]!=s[j]) j=p[j-1];
10            p[i]=j+(s[i]==s[j]);
11        }
12        return p;
13    }
14    static vector<int> match(const string &s,const string &t)
15    {
16        assert(!t.empty()); auto p=prefix(t); vector<int> ans;
17        for (int i=0,j=0;i<(int)s.size();i++)
18        {
19            while (j&& s[i]!=t[j]) j=p[j-1];
20            j+=(s[i]==t[j]);
21            if (j==(int)t.size()) { ans.push_back(i-j+1); j=p[j-1]; }
22        }
23        return ans;
24    }
25    static vector<int> z(const string &s)
26    {
27        int n=(int)s.size(); vector<int> a(n);
28        if (n) a[0]=n;
29        for (int i=1,l=0,r=0;i<n;i++)
30        {
31            if (i<r) a[i]=min(r-i,a[i-1]);
32            while (i+a[i]<n&& s[a[i]]==s[i+a[i]]) ++a[i];
33            if (i+a[i]>r) { l=i; r=i+a[i]; }
34        }
35        return a;
36    }
37    // odd[i]: radius including center; even[i]: pairs centered before i.
38    static pair<vector<int>,vector<int>> manacher(const string &s)
39    {
40        int n=(int)s.size(); vector<int> odd(n),even(n);
41        for (int i=0,l=0,r=-1;i<n;i++)
42        {
43            int k=i>r?1:min(odd[l+r-i],r-i+1);
44            while (i-k>=0&&i+k<n&& s[i-k]==s[i+k]) ++k;
45            odd[i]=k--;
46            if (i+k>r) { l=i-k; r=i+k; }
47        }
48        for (int i=0,l=0,r=-1;i<n;i++)
49        {
50            int k=i>r?0:min(even[l+r-i+1],r-i+1);
51            while (i-k-1>=0&&i+k<n&& s[i-k-1]==s[i+k]) ++k;
52            even[i]=k--;
53            if (i+k>r) { l=i-k-1; r=i+k; }
54        }
55        return {odd,even};
56    }
57    static int rotation(const string &s)
58    {
```

```

59     int n=(int)s.size(),i=0,j=1,k=0;
60     if (!n) return 0;
61     while (i<n&&j<n&&k<n)
62     {
63         unsigned char a=s[(i+k)%n],b=s[(j+k)%n];
64         if (a==b) { ++k; continue; }
65         if (a>b) { i+=k+1; if (i==j) ++i; }
66         else { j+=k+1; if (i==j) ++j; }
67         k=0;
68     }
69     return min(i,j);
70 }
71 };

```

5.2 AC 自动机出现次数

字母表 a..z, add(s) 返回终止状态, 要求模式非空; 全部插入后 build(), 只允许一次构建。count(text) 返回每个状态对应模式的出现次数, 可重复查询; 重复模式共享状态。构建 $O(26N)$, 查询 $O(|text|+N)$ 。

```

1 struct AhoCorasick
2 {
3     struct Node { array<int,26> go{}; int fail=0; };
4     vector<Node> a{Node{}}; vector<int> order; bool built=false;
5     int add(const string &s)
6     {
7         assert(!built&&!s.empty()); int u=0;
8         for (char ch:s)
9         {
10             int c=ch-'a'; assert(0<=c&&c<26);
11             if (!a[u].go[c]) { int v=(int)a.size(); a[u].go[c]=v; a.push_back(Node{ }); }
12             u=a[u].go[c];
13         }
14         return u;
15     }
16     void build()
17     {
18         assert(!built); built=true; queue<int> q;
19         for (int v:a[0].go) if (v) q.push(v);
20         while (!q.empty())
21         {
22             int u=q.front(); q.pop(); order.push_back(u);
23             for (int c=0;c<26;c++)
24             {
25                 int v=a[u].go[c];
26                 if (v) { a[v].fail=a[a[u].fail].go[c]; q.push(v); }
27                 else a[u].go[c]=a[a[u].fail].go[c];
28             }
29         }
30     }
31     vector<long long> count(const string &s) const
32     {
33         assert(built); vector<long long> ans(a.size()); int u=0;
34         for (char ch:s) { u=a[u].go[ch-'a']; ++ans[u]; }
35         for (int i=(int)order.size()-1;i>=0;i--) ans[a[order[i]].fail]+=ans[order[i]];
36         return ans;
37     }
38 };

```

5.3 倍增后缀数组与 LCP

$sa[k]$ 为第 k 个后缀起点, $rk[i]$ 为排名, $lcp[k]$ 为 $sa[k-1]$ 与 $sa[k]$ 的 LCP ($lcp[0]=0$)。空串合法; 当前排序比较版 $O(n \log^2 n)$, 不能替代待补的线性 SA/DC3 条目。

```

1 struct SuffixArray
2 {
3     vector<int> sa,rk,lcp;
4     SuffixArray(const string &s)
5     {
6         int n=(int)s.size(); sa.resize(n); rk.resize(n); lcp.assign(n,0);
7         for (int i=0;i<n;i++) { sa[i]=i; rk[i]=(unsigned char)s[i]; }
8         vector<int> tmp(n);
9         for (int k=1;k<n;k*=2)
10        {
11            auto key=[&](int i) { return pair{rk[i],i+k<n?rk[i+k]:-1}; };
12            sort(sa.begin(),sa.end(),[&](int i,int j) { return key(i)<key(j); });
13            tmp[sa[0]]=0;
14            for (int i=1;i<n;i++) tmp[sa[i]]=tmp[sa[i-1]]+(key(sa[i])!=key(sa[i-1]));
15            rk=tmp; if (rk[sa.back()]==n-1) break;
16        }
17        for (int i=0;i<n;i++) rk[sa[i]]=i;
18        for (int i=0,k=0;i<n;i++)
19        {
20            if (!rk[i]) { k=0; continue; }
21            int j=sa[rk[i]-1];
22            while (i+k<n&&j+k<n&&s[i+k]==s[j+k]) ++k;
23            lcp[rk[i]]=k; if (k) --k;
24        }
25    }
26 };

```

5.4 后缀自动机

`extend(c)` 接收整数 0..25; `distinct()` 返回不同非空子串数, `counts()` 返回 `endpos` 次数且不修改结构, 可重复调用。状态 `a`, 初始状态 0, `link` 根为 -1; 空间 $O(26n)$, 构建 $O(26n)$ 。

```

1 struct SuffixAutomaton
2 {
3     struct Node { array<int,26> go{}; int link=-1,len=0; long long occ=0; };
4     vector<Node> a{Node{}}; int last=0;
5     void extend(int c)
6     {
7         assert(0<=c&&c<26); int cur=(int)a.size(); a.push_back(Node{});
8         a[cur].len=a[last].len+1; a[cur].occ=1; int p=last;
9         while (p!=-1&&a[p].go[c]) { a[p].go[c]=cur; p=a[p].link; }
10        if (p==-1) a[cur].link=0;
11        else
12        {
13            int q=a[p].go[c];
14            if (a[p].len+1==a[q].len) a[cur].link=q;
15            else
16            {
17                int clone=(int)a.size(); Node copy=a[q]; a.push_back(copy);
18                a[clone].len=a[p].len+1; a[clone].occ=0;
19                while (p!=-1&&a[p].go[c]==q) { a[p].go[c]=clone; p=a[p].link; }
20                a[q].link=a[cur].link=clone;
21            }
22        }
23        last=cur;
24    }
25    vector<long long> counts() const
26    {
27        vector<int> bucket(a[last].len+1,order(a.size())); vector<long long> ans(a.size());
28        for (auto &v:a) ++bucket[v.len];
29        partial_sum(bucket.begin(),bucket.end(),bucket.begin());
30        for (int i=0;i<(int)a.size();i++) { order[--bucket[a[i].len]]=i; ans[i]=a[i].occ; }
31        for (int i=(int)order.size()-1;i>0;i--) ans[a[order[i]].link]+=ans[order[i]];
32        return ans;
33    }
34    long long distinct() const
35    {
36        long long ans=0;
37        for (int i=1;i<(int)a.size();i++) ans+=a[i].len-a[a[i].link].len;
38        return ans;
39    }
40 };

```

第六章 数论

6.1 模运算、素性、CRT 与类欧几里德

mul/power 支持无符号 64 位正模数，乘积用 uint128。prime 使用确定性 64 位底数。inverse(a,m) 无逆元返回 -1。crt(r,m,b,n) 合并同余式，无解返回 false，模数溢出抛异常。floor_sum 为 $i=0..n-1$ 的 $\text{floor}((ai+b)/m)$ 之和，支持负 a、b，返回 int128；约定 $n \leq 10^9$ 、 $m \leq 10^9$ ，a、b 为有符号 64 位。

```
1 struct NumberTheory
2 {
3     using ll=long long; using ull=unsigned long long;
4     using i128=__int128_t; using u128=__uint128_t;
5     static ull mul(ull a,ull b,ull m) { return u128(a)*b%m; }
6     static ull power(ull a,ull b,ull m)
7     {
8         assert(m); ull r=1%m; a%=m;
9         for (;b>=1,a=mul(a,a,m)) if (b&1) r=mul(r,a,m);
10        return r;
11    }
12    static bool prime(ull n)
13    {
14        if (n<2) return false;
15        for (ull p:{2,3,5,7,11,13,17,19,23,29,31,37})
16            if (n%p==0) return n==p;
17        ull d=n-1; int s=0;
18        while (!(d&1)) { d>>=1; ++s; }
19        for (ull a:{2ULL,325ULL,9375ULL,28178ULL,450775ULL,9780504ULL,1795265022ULL})
20        {
21            if (a%n==0) continue;
22            ull x=power(a,d,n);
23            if (x==1||x==n-1) continue;
24            bool ok=false;
25            for (int j=1;j<s;j++) { x=mul(x,x,n); if (x==n-1) { ok=true; break; } }
26            if (!ok) return false;
27        }
28        return true;
29    }
30    static i128 exgcd(i128 a,i128 b,i128 &x,i128 &y)
31    {
32        if (!b) { x=1; y=0; return a; }
33        i128 d=exgcd(b,a%b,y,x); y-=a/b*x; return d;
34    }
35    static ll inverse(ll a,ll m)
36    {
37        assert(m>0); a%=m; if (a<0) a+=m;
38        i128 x,y; if (exgcd(a,m,x,y)!=1) return -1;
39        return (x%m+m)%m;
40    }
41    // x = r (mod m); false means inconsistent. Throws if lcm exceeds int64.
42    static bool crt(ll &r,ll &m,ll b,ll n)
43    {
44        assert(m>0&&n>0); r%=m; if (r<0) r+=m; b%=n; if (b<0) b+=n;
45        i128 x,y; ll g=(ll)exgcd(m,n,x,y); i128 diff=i128(b)-r;
46        if (diff%g) return false;
47        i128 q=n/g,k=(diff/g*x+q)%q,mod=i128(m)*q;
48        if (mod>LLONG_MAX) throw overflow_error("CRT modulus");
49        r=(r+i128(m)*k)%mod; m=(ll)mod; return true;
50    }
51    // sum_{i=0}^{n-1} floor((a*i+b)/m), signed a,b; 0<=n<=1e9, 1<=m<=1e9.
52    static i128 floor_sum(ll n,ll m,ll a,ll b)
53    {
54        assert(n>=0&&m>0); i128 ans=0;
55        auto norm=[&](ll &v)
56        { ll q=v/m; if (v%m<0) --q; v=(ll)(i128(v)-i128(q)*m); return q; };
```

```

57     ans+=i128(norm(a))*n*(n-1)/2; ans+=i128(norm(b))*n;
58     while (true)
59     {
60         ans+=i128(n)*(n-1)/2*(a/m); a%=m;
61         ans+=i128(n)*(b/m); b%=m;
62         i128 y=i128(a)*n+b;
63         if (y<m) break;
64         n=(ll)(y/m); b=(ll)(y%m); swap(a,m);
65     }
66     return ans;
67 }
68 };

```

6.2 Pollard–Rho 因数分解

`factor(n)` 返回升序质因子（含重数）， $n \geq 1$ ；依赖 `NumberTheory`。返回因子经过整除和素性检查；Las Vegas 随机运行时间，固定默认种子方便复现，可由调用者更换。无确定性最坏时间保证。

```

1 struct PollardRho
2 {
3     using ull=unsigned long long; using u128=__uint128_t;
4     mt19937_64 rng;
5     PollardRho(ull seed=712367821):rng(seed) {}
6     ull rho(ull n)
7     {
8         if (n%2==0) return 2;
9         for (;;)
10         {
11             ull c=rng()%(n-1)+1,x=rng()%n,y=x,d=1;
12             auto f=[&](ull v) { return (NumberTheory::mul(v,v,n)+u128(c))%n; };
13             // Retry bounded attempts; Las Vegas: only exact divisors returned.
14             for (int i=0;i<200000&&d==1;i++)
15                 { x=f(x); y=f(f(y)); d=gcd(x>y?x-y:y-x,n); }
16             if (1<d&&d<n) return d;
17         }
18     }
19     void split(ull n,vector<ull> &a)
20     {
21         if (n==1) return;
22         if (NumberTheory::prime(n)) { a.push_back(n); return; }
23         ull d=rho(n); split(d,a); split(n/d,a);
24     }
25     vector<ull> factor(ull n)
26     { assert(n>=1); vector<ull> a; split(n,a); sort(a.begin(),a.end()); return a; }
27 };

```

6.3 线性筛、欧拉函数与 Möbius 函数

构造参数 $n \geq 0$, prime 为质数表, lp/phi/mu 为 $0..n$ 数组。phi[1]=mu[1]=1, $O(n)$ 时间和空间。

```

1 struct LinearSieve
2 {
3     vector<int> prime,lp,phi,mu;
4     LinearSieve(int n):lp(n+1),phi(n+1),mu(n+1)
5     {
6         if (n) phi[1]=mu[1]=1;
7         for (int i=2;i<=n;i++)
8         {
9             if (!lp[i]) { lp[i]=i; prime.push_back(i); phi[i]=i-1; mu[i]=-1; }
10            for (int p:prime)
11            {
12                if (p>n/i) break;
13                int j=i*p; lp[j]=p;
14                if (i%p==0) { phi[j]=phi[i]*p; mu[j]=0; break; }
15                phi[j]=phi[i]*(p-1); mu[j]=-mu[i];
16            }
17        }
18    }
19 };

```

6.4 静态模整数

模板参数必须为不超过 INT_MAX 的素数且 ≥ 2 。加减乘、pow、inv、除法；求逆要求非零，pow 指数非负。
复杂度：加减乘 $O(1)$ ，幂和逆 $O(\log \text{mod})$ 。

```

1 template<int mod> struct ModInt
2 {
3     int v;
4     ModInt(long long x=0):v((x%mod+mod)%mod) {}
5     ModInt operator+(ModInt b) const { return ModInt((long long)v+b.v); }
6     ModInt operator-(ModInt b) const { return ModInt((long long)v-b.v); }
7     ModInt operator*(ModInt b) const { return ModInt(1LL*v*b.v); }
8     ModInt pow(long long e) const
9     { assert(e>=0); ModInt a=*this,r=1; for (;e>=1;a=a*a) if (e&1) r=r*a; return r; }
10    ModInt inv() const { assert(v); return pow(mod-2); } // prime modulus
11    ModInt operator/(ModInt b) const { return *this*b.inv(); }
12 };

```

6.5 阶乘组合数

Binomial<prime>(n), 要求 $0 \leq n < \text{prime}$; choose(n,k) 越界 k 返回 0。预处理 $O(n + \log \text{prime})$, 单次 $O(1)$ 。
尚不能替代合数模数 exLucas。

```

1 template<int mod> struct Binomial
2 {
3     using Z=ModInt<mod>; vector<Z> fac,ifac;
4     Binomial(int n):fac(n+1),ifac(n+1)
5     {
6         assert(0<=n&&n<mod); fac[0]=1;
7         for (int i=1;i<=n;i++) fac[i]=fac[i-1]*i;
8         ifac[n]=fac[n].inv();
9         for (int i=n;i-->0) ifac[i-1]=ifac[i]*i;
10    }
11    Z choose(int n,int k) const
12    { if (k<0||k>n) return 0; assert(n<(int)fac.size()); return fac[n]*ifac[k]*ifac[n-k]; }
13 };

```

第七章 多项式

7.1 NTT、形式幂级数、FWT 与递推

依赖 number_theory.hpp。模数固定 998244353，乘法结果长度 $\leq 2^{23}$ ；inverse 要求 $a[0] \neq 0$ ，log 要求 $a[0]=1$ ，exp 要求 $a[0]=0$ ，截断长度 $n>0$ 。朴素逐层 NTT 实现的 FPS 为 $O(n \log n)$ 。FWT 的 op 为 $\wedge$ 、 $|$ 、 $\&$ ；BM 返回 $s[n]=\sum c[i-1]s[n-i]$ 的系数；recurrence 用 $O(k^2 \log n)$ 求第 n 项。BM 对有限样本给出的递推不自动保证未知后续项成立。

```
1 struct Polynomial
2 {
3     static constexpr int mod=998244353;
4     using Z=ModInt<mod>; using Poly=vector<Z>;
5     static void ntt(Poly &a,bool invert=false)
6     {
7         int n=(int)a.size(); assert(n>0&&(n&(n-1))==0&&n<=(1<<23));
8         for (int i=1,j=0;i<n;i++)
9         {
10             int bit=n>>1; for (;j&bit;bit>>=1) j^=bit; j^=bit;
11             if (i<j) swap(a[i],a[j]);
12         }
13         for (int len=2;len<=n;len*=2)
14         {
15             Z step=Z(3).pow((mod-1)/len); if (invert) step=step.inv();
16             for (int i=0;i<n;i+=len)
17             {
18                 Z w=1;
19                 for (int j=0;j<len/2;j++,w=w*step)
20                 {
21                     Z u=a[i+j],v=a[i+j+len/2]*w;
22                     a[i+j]=u+v; a[i+j+len/2]=u-v;
23                 }
24             }
25         }
26         if (invert) { Z inv=Z(n).inv(); for (auto &x:a) x=x*inv; }
27     }
28     static Poly multiply(Poly a,Poly b)
29     {
30         if (a.empty()||b.empty()) return {};
31         int size=(int)a.size()+(int)b.size()-1,n=1;
32         assert(size<=(1<<23)); while (n<size) n*=2;
33         a.resize(n); b.resize(n); ntt(a); ntt(b);
34         for (int i=0;i<n;i++) a[i]=a[i]*b[i];
35         ntt(a,true); a.resize(size); return a;
36     }
37     static Poly inverse(const Poly &a,int n)
38     {
39         assert(n>0&&!a.empty()&&a[0].v); Poly b{a[0].inv()};
40         while ((int)b.size()<n)
41         {
42             int m=min(n,2*(int)b.size());
43             Poly f(a.begin(),a.begin()+min(m,(int)a.size()));
44             auto t=multiply(f,b); t.resize(m);
45             for (auto &x:t) x=Z(0)-x;
46             t[0]=t[0]+2;
47             b=multiply(b,t); b.resize(m);
48         }
49         return b;
50     }
51     static Poly derivative(const Poly &a)
52     { Poly b; for (int i=1;i<(int)a.size();i++) b.push_back(a[i]*i); return b; }
53     static Poly integral(const Poly &a)
54     {
55         Poly b(a.size()+1); vector<Z> inv(a.size()+1);
```

```

56     if (!a.empty()) inv[1]=1;
57     for (int i=2;i<=(int)a.size();i++) inv[i]=Z(mod-mod/i)*inv[mod%i];
58     for (int i=0;i<(int)a.size();i++) b[i+1]=a[i]*inv[i+1];
59     return b;
60 }
61 static Poly log(const Poly &a,int n)
62 {
63     assert(n>0&&!a.empty()&&a[0].v==1);
64     auto b=multiply(derivative(a),inverse(a,n)); b.resize(n-1);
65     return integral(b);
66 }
67 static Poly exp(const Poly &a,int n)
68 {
69     assert(n>0&&(a.empty()||a[0].v==0)); Poly b{1};
70     while ((int)b.size()<n)
71     {
72         int m=min(n,2*(int)b.size()); auto t=log(b,m);
73         for (int i=0;i<m;i++) t[i]=(i<(int)a.size()?a[i]:Z(0))-t[i];
74         t[0]=t[0]+1; b=multiply(b,t); b.resize(m);
75     }
76     return b;
77 }
78 static void fwt(Poly &a,char op,bool invert=false)
79 {
80     int n=(int)a.size(); assert(n>0&&(n&(n-1))==0);
81     const Z inv2=(mod+1)/2;
82     for (int len=1;len<n;len*=2)
83         for (int i=0;i<n;i+=len*2)
84             for (int j=0;j<len;j++)
85             {
86                 Z &u=a[i+j],&v=a[i+j+len],x=u,y=v;
87                 if (op=='^') { u=x+y; v=x-y; if (invert) { u=u*inv2; v=v*inv2; } }
88                 else if (op=='|') v=invert?y-x:y+x;
89                 else { assert(op=='&'); u=invert?x-y:x+y; }
90             }
91 }
92 // c: s[n]=sum c[i-1]*s[n-i], i=1..L.
93 static Poly berlekamp_massey(const Poly &s)
94 {
95     Poly c{1},b{1}; int l=0,m=1; Z last=1;
96     for (int n=0;n<(int)s.size();n++)
97     {
98         Z d=s[n]; for (int i=1;i<=l;i++) d=d+c[i]*s[n-i];
99         if (!d.v) { ++m; continue; }
100         auto old=c; Z coef=d/last;
101         if (c.size()<b.size()+m) c.resize(b.size()+m);
102         for (int j=0;j<(int)b.size();j++) c[j+m]=c[j+m]-coef*b[j];
103         if (2*l<=n) { l=n+1-l; b=old; last=d; m=1; }
104         else ++m;
105     }
106     c.erase(c.begin()); for (auto &x:c) x=Z(0)-x; return c;
107 }
108 static Z recurrence(Poly init,const Poly &c,unsigned long long n)
109 {
110     int k=(int)c.size(); if (n<init.size()) return init[n];
111     if (!k) return 0;
112     assert((int)init.size()>=k);
113     auto combine=[&](const Poly &a,const Poly &b)
114     {
115         Poly t(2*k-1);
116         for (int i=0;i<k;i++) for (int j=0;j<k;j++) t[i+j]=t[i+j]+a[i]*b[j];
117         for (int i=2*k-2;i>=k;i--)
118             for (int j=1;j<=k;j++) t[i-j]=t[i-j]+t[i]*c[j-1];
119         t.resize(k); return t;
120     };
121     Poly a(k),r(k); r[0]=1; if (k==1) a[0]=c[0]; else a[1]=1;
122     for (;n>=1,a=combine(a,a)) if (n&1) r=combine(r,a);
123     Z ans=0; for (int i=0;i<k;i++) ans=ans+r[i]*init[i]; return ans;
124 }
125 };

```

第八章 代数与数论进阶

8.1 模高斯消元、行列式、矩阵树定理

素数模数。solve(a,n) 输入增广矩阵，返回 consistent、rank、特解 particular 和零空间基 kernel。determinant(empty)=1。spanning_trees(n,edges) 使用 0-based 无向图，允许重边，忽略自环；返回生成树数模 p。高斯与行列式 $O(n^3)$ ，矩阵幂 $O(n^3 \log e)$ 。

```
1 template<int mod> struct LinearAlgebra
2 {
3     using Z=ModInt<mod>; using Matrix=vector<vector<Z>>;
4     struct Solution { bool consistent; int rank; vector<Z> particular; Matrix kernel; };
5     // Augmented m x (n+1), prime modulus. Empty system needs explicit n.
6     static Solution solve(Matrix a,int n)
7     {
8         int m=(int)a.size(),row=0; vector<int> where(n,-1);
9         for (const auto &v:a) assert((int)v.size()==n+1);
10        for (int col=0;col<n&&row<m;col++)
11        {
12            int p=row; while (p<m&&!a[p][col].v) ++p;
13            if (p==m) continue;
14            swap(a[p],a[row]); Z inv=a[row][col].inv();
15            for (int j=col;j<=n;j++) a[row][j]=a[row][j]*inv;
16            for (int i=0;i<m;i++) if (i!=row&&a[i][col].v)
17            {
18                Z f=a[i][col];
19                for (int j=col;j<=n;j++) a[i][j]=a[i][j]-f*a[row][j];
20            }
21            where[col]=row++;
22        }
23        for (int i=row;i<m;i++) if (a[i][n].v) return {false,row,{},{}};
24        Solution ans{true,row,vector<Z>(n),{}};
25        for (int j=0;j<n;j++) if (where[j]!=-1) ans.particular[j]=a[where[j]][n];
26        for (int j=0;j<n;j++) if (where[j]==-1)
27        {
28            vector<Z> v(n); v[j]=1;
29            for (int k=0;k<n;k++) if (where[k]!=-1) v[k]=Z(0)-a[where[k]][j];
30            ans.kernel.push_back(v);
31        }
32        return ans;
33    }
34    static Z determinant(Matrix a)
35    {
36        int n=(int)a.size(); Z ans=1;
37        for (int i=0;i<n;i++)
38        {
39            assert((int)a[i].size()==n); int p=i;
40            while (p<n&&!a[p][i].v) ++p;
41            if (p==n) return 0;
42            if (p!=i) { swap(a[p],a[i]); ans=Z(0)-ans; }
43            ans=ans*a[i][i]; Z inv=a[i][i].inv();
44            for (int j=i+1;j<n;j++)
45            {
46                Z f=a[j][i]*inv;
47                for (int k=i;k<n;k++) a[j][k]=a[j][k]-f*a[i][k];
48            }
49        }
50        return ans;
51    }
52    static Matrix multiply(const Matrix &a,const Matrix &b)
53    {
54        assert(!a.empty()&&!b.empty()); int n=a.size(),m=b[0].size(),k=b.size();
55        assert((int)a[0].size()==k); Matrix c(n,vector<Z>(m));
56        for (int i=0;i<n;i++) for (int t=0;t<k;t++)
```

```

57         for (int j=0;j<m;j++) c[i][j]=c[i][j]+a[i][t]*b[t][j];
58     return c;
59 }
60 static Matrix power(Matrix a,unsigned long long e)
61 {
62     int n=a.size(); assert(n>0&&(int)a[0].size()==n);
63     Matrix r(n,vector<Z>(n)); for (int i=0;i<n;i++) r[i][i]=1;
64     for (;e>=1,a=multiply(a,a)) if (e&1) r=multiply(r,a);
65     return r;
66 }
67 // Undirected multigraph, vertices 0..n-1; loops ignored.
68 static Z spanning_trees(int n,const vector<pair<int,int>> &edges)
69 {
70     assert(n>0); Matrix lap(n,vector<Z>(n));
71     for (auto [u,v]:edges) if (u!=v)
72     { lap[u][u]=lap[u][u]+1; lap[v][v]=lap[v][v]+1; lap[u][v]=lap[u][v]-1; lap[v][u]=lap[v][u]-1; }
73     lap.pop_back(); for (auto &row:lap) row.pop_back(); return determinant(lap);
74 }
75 };

```

8.2 杜教筛

构造预筛上限 $B \geq 1$, $\text{mertens}(n)$ 返回 μ 的前缀和, $\text{totient_sum}(n)$ 返回 φ 的前缀和 (int128)。约定 $0 \leq n \leq 10^{11}$, 按整除分块递归并缓存; 大输入需要合理 B , 不能把常数 B 当作性能保证。

```

1 struct DuJiao
2 {
3     using ll=long long; using I=__int128_t;
4     int limit; vector<ll> pmu,pphi;
5     unordered_map<ll,ll> mmu; unordered_map<ll,I> mphi;
6     DuJiao(int limit):limit(limit),pmu(limit+1),pphi(limit+1)
7     {
8         assert(limit>=1); LinearSieve s(limit);
9         for (int i=1;i<=limit;i++) { pmu[i]=pmu[i-1]+s.mu[i]; pphi[i]=pphi[i-1]+s.phi[i]; }
10    }
11    ll mertens(ll n)
12    {
13        if (n<=limit) return pmu[n];
14        if (mmu.count(n)) return mmu[n];
15        ll ans=1;
16        for (ll l=2,r;l<=n;l=r+1) { r=n/(n/l); ans-=(r-l+1)*mertens(n/l); }
17        return mmu[n]=ans;
18    }
19    I totient_sum(ll n)
20    {
21        if (n<=limit) return pphi[n];
22        if (mphi.count(n)) return mphi[n];
23        I ans=I(n)*(n+1)/2;
24        for (ll l=2,r;l<=n;l=r+1) { r=n/(n/l); ans-=I(r-l+1)*totient_sum(n/l); }
25        return mphi[n]=ans;
26    }
27 };

```

8.3 扩展 BSGS

$\text{solve}(a,b,m)$ 返回最小 $x \geq 0$ 使 $a^x \equiv b \pmod{m}$, 无解 -1; $1 \leq m \leq 10^{12}$, $0^0 = 1$ 。时间与内存 $O(\sqrt{m})$, unordered_map 复杂度为期望界。

```

1 struct DiscreteLog
2 {
3     using ll=long long;
4     // 1<=m<=1e12; O(sqrt(m)) time/storage. Smallest x>=0, or -1.
5     static ll solve(ll a,ll b,ll m)
6     {
7         assert(m>=1&&m<=1000000000000LL); a=(a%m+m)%m; b=(b%m+m)%m;
8         if (m==1) return 0;
9         ll offset=0,k=1;
10        for (ll g;(g=gcd(a,m))>1;)
11        {
12            if (b==k) return offset;
13            if (b%g) return -1;
14            b/=g; m/=g; k=(__int128)k*(a/g)%m; ++offset;
15        }
16        ll target=(__int128)b*NumberTheory::inverse(k,m)%m;
17        ll step=sqrtl(m)+1; unordered_map<ll,ll> baby;
18        ll cur=1%m;
19        for (ll j=0;j<step;j++) { if (!baby.count(cur)) baby[cur]=j; cur=(__int128)cur*a%m; }
20        ll inv=NumberTheory::inverse(NumberTheory::power(a,step,m),m); cur=target;
21        for (ll i=0;i<=step;i++)
22        {
23            auto it=baby.find(cur); if (it!=baby.end()) return offset+i*step+it->second;
24            cur=(__int128)cur*inv%m;
25        }
26        return -1;
27    }
28 };

```

第九章 计算几何

9.1 整数精确二维几何

坐标绝对值 $\leq 10^{12}$ ，点数 $\leq 10^6$ 。叉积、点积、线段相交、凸包、点在多边形、点在凸包、旋转卡壳直径。凸包逆时针，不保留共线内点，不重复首点。contains 返回 0 外部、1 边界、2 内部。convex_contains/diameter2 输入必须为本模板 strict hull。PolarLess 要求排除零向量。

```
1 struct IntegerGeometry
2 {
3     using ll=long long; using I=__int128_t;
4     // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
5     struct Point
6     {
7         ll x=0,y=0;
8         bool operator<(Point b) const { return tie(x,y)<tie(b.x,b.y); }
9         bool operator==(Point b) const { return x==b.x&&y==b.y; }
10    };
11    static I cross(Point a,Point b,Point c)
12    { return (I(b.x)-a.x)*(I(c.y)-a.y)-(I(b.y)-a.y)*(I(c.x)-a.x); }
13    static I dot(Point a,Point b,Point c)
14    { return (I(b.x)-a.x)*(I(c.x)-a.x)+(I(b.y)-a.y)*(I(c.y)-a.y); }
15    static I dist2(Point a,Point b) { return dot(a,b,b); }
16    static int sign(I x) { return (x>0)-(x<0); }
17    static bool on_segment(Point p,Point a,Point b)
18    { return cross(a,b,p)==0&&dot(p,a,b)<=0; }
19    static bool intersect(Point a,Point b,Point c,Point d)
20    {
21        int x=sign(cross(a,b,c)),y=sign(cross(a,b,d));
22        int z=sign(cross(c,d,a)),w=sign(cross(c,d,b));
23        if (!x&&on_segment(c,a,b)) return true;
24        if (!y&&on_segment(d,a,b)) return true;
25        if (!z&&on_segment(a,c,d)) return true;
26        if (!w&&on_segment(b,c,d)) return true;
27        return x*y<0&&z*w<0;
28    }
29    static vector<Point> hull(vector<Point> p)
30    {
31        sort(p.begin(),p.end()); p.erase(unique(p.begin(),p.end()),p.end());
32        int n=(int)p.size(); if (n<3) return p;
33        vector<Point> h;
34        for (Point v:p)
35        {
36            while (h.size()>1&&cross(h[h.size()-2],h.back(),v)<=0) h.pop_back();
37            h.push_back(v);
38        }
39        int low=(int)h.size();
40        for (int i=n-2;i>=0;i--)
41        {
42            while ((int)h.size()>low&&cross(h[h.size()-2],h.back(),p[i])<=0) h.pop_back();
43            h.push_back(p[i]);
44        }
45        h.pop_back(); return h; // CCW, no repeated endpoint, no collinear interior points.
46    }
47    static I area2(const vector<Point> &p)
48    {
49        I ans=0; int n=(int)p.size();
50        for (int i=0;i<n;i++) ans+=I(p[i].x)*p[(i+1)%n].y-I(p[i].y)*p[(i+1)%n].x;
51        return ans;
52    }
53    // 0 outside, 1 boundary, 2 inside. Simple polygon; either orientation.
54    static int contains(const vector<Point> &p,Point q)
55    {
56        int winding=0,n=(int)p.size();
```

```

57     for (int i=0;i<n;i++)
58     {
59         Point a=p[i],b=p[(i+1)%n];
60         if (on_segment(q,a,b)) return 1;
61         I c=cross(a,b,q);
62         if (a.y<=q.y&& b.y>q.y&& c>0) ++winding;
63         if (a.y>q.y&& b.y<=q.y&& c<0) --winding;
64     }
65     return winding?2:0;
66 }
67 // Input must be the strict CCW hull above. O(log n).
68 static int convex_contains(const vector<Point> &p,Point q)
69 {
70     int n=(int)p.size(); if (n<3) return contains(p,q);
71     I a=cross(p[0],p[1],q),b=cross(p[0],p[n-1],q);
72     if (a<0||b>0) return 0;
73     if (!a) return on_segment(q,p[0],p[1])?1:0;
74     if (!b) return on_segment(q,p[0],p[n-1])?1:0;
75     int l=1,r=n-1;
76     while (l+1<r) { int m=(l+r)/2; if (cross(p[0],p[m],q)>=0) l=m; else r=m; }
77     I c=cross(p[1],p[r],q); return c<0?0:c==0?1:2;
78 }
79 static I diameter2(const vector<Point> &p)
80 {
81     int n=(int)p.size(); if (n<2) return 0;
82     if (n==2) return dist2(p[0],p[1]);
83     I ans=0; int j=1;
84     for (int i=0;i<n;i++)
85     {
86         int k=(i+1)%n;
87         while (cross(p[i],p[k],p[(j+1)%n])>cross(p[i],p[k],p[j])) j=(j+1)%n;
88         ans=max({ans,dist2(p[i],p[j]),dist2(p[k],p[j])});
89     }
90     return ans;
91 }
92 struct PolarLess
93 {
94     static int half(Point p) { return p.y<0||(p.y==0&&p.x<0); }
95     bool operator()(Point a,Point b) const
96     {
97         if (half(a)!=half(b)) return half(a)<half(b);
98         I c=cross({0,0},a,b);
99         if (c) return c>0;
100         return dist2({0,0},a)<dist2({0,0},b);
101     }
102 };
103 };

```

9.2 浮点点线圆构造

long double, 显式 eps 默认 10^{-12} 。要求有限输入及适当尺度；接近退化时按容差分类，不能提供任意实数输入的精确拓扑保证。Result 区分 none/one/two/infinite/degenerate。零长度线返回 degenerate；投影与点到线段距离将零线段视为点。坐标缩放和误差预算应按具体题面复核。

```

1 struct RealGeometry
2 {
3     using R=long double;
4     // Tolerance is explicit, not used in sorting. Requires finite, well-scaled input.
5     R eps;
6     RealGeometry(R eps=1e-12L):eps(eps) { assert(eps>0); }
7     struct Point
8     {
9         R x=0,y=0;
10        Point operator+(Point b) const { return {x+b.x,y+b.y}; }
11        Point operator-(Point b) const { return {x-b.x,y-b.y}; }
12        Point operator*(R k) const { return {x*k,y*k}; }
13        Point operator/(R k) const { return {x/k,y/k}; }
14    };

```

```

15 struct Circle { Point o; R r; };
16 enum class Kind { none, one, two, infinite, degenerate };
17 struct Result { Kind kind; vector<Point> p; };
18 static R dot(Point a, Point b) { return a.x*b.x+a.y*b.y; }
19 static R cross(Point a, Point b) { return a.x*b.y-a.y*b.x; }
20 static R norm(Point a) { return hypotl(a.x, a.y); }
21 static Point perp(Point a) { return {-a.y, a.x}; }
22 Point projection(Point p, Point a, Point b) const
23 {
24     Point v=b-a; R d=dot(v,v);
25     if (d==0) return a;
26     return a+v*(dot(p-a,v)/d);
27 }
28 R segment_distance(Point p, Point a, Point b) const
29 {
30     Point v=b-a; R d=dot(v,v);
31     if (d==0) return norm(p-a);
32     R t=clamp(dot(p-a,v)/d, R(0), R(1)); return norm(p-(a+v*t));
33 }
34 Result line_intersection(Point a, Point b, Point c, Point d) const
35 {
36     Point u=b-a, v=d-c; R nu=norm(u), nv=norm(v);
37     if (nu==0 || nv==0) return {Kind::degenerate, {}};
38     R det=cross(u, v);
39     if (fabsl(det)<=eps*nu*nv)
40         return {fabsl(cross(c-a, u))<=eps*nu*max(R(1), norm(c-a))?Kind::infinite:Kind::none, {}};
41     return {Kind::one, {a+u*(cross(c-a, v)/det)}};
42 }
43 Result line_circle(Point a, Point b, Circle c) const
44 {
45     assert(c.r>=0); Point v=b-a; R len=norm(v);
46     if (len==0) return {Kind::degenerate, {}};
47     Point h=projection(c.o, a, b); R d=norm(h-c.o), tol=eps*max({R(1), d, c.r});
48     if (d>c.r+tol) return {Kind::none, {}};
49     if (fabsl(d-c.r)<=tol) return {Kind::one, {h}};
50     R t=sqrtl(max(R(0), (c.r-d)*(c.r+d))); Point w=v*(t/len);
51     return {Kind::two, {h-w, h+w}};
52 }
53 Result circles(Circle a, Circle b) const
54 {
55     assert(a.r>=0 && b.r>=0); Point v=b.o-a.o; R d=norm(v);
56     R tol=eps*max({R(1), d, a.r, b.r});
57     if (d<=tol)
58     {
59         if (fabsl(a.r-b.r)>tol) return {Kind::none, {}};
60         if (a.r==0 && b.r==0) return {Kind::one, {a.o}};
61         return {Kind::infinite, {}};
62     }
63     if (d>a.r+b.r+tol || d<fabsl(a.r-b.r)-tol) return {Kind::none, {}};
64     R x=(d*(d+(a.r-b.r)*(a.r+b.r))/(2*d);
65     Point h=a.o+v*(x/d);
66     if (fabsl(d-(a.r+b.r))<=tol || fabsl(d-fabsl(a.r-b.r))<=tol)
67         return {Kind::one, {h}};
68     R y=sqrtl(max(R(0), (a.r-x)*(a.r+x))); Point w=perp(v)*(y/d);
69     return {Kind::two, {h-w, h+w}};
70 }
71 R overlap(Circle a, Circle b) const
72 {
73     R d=norm(a.o-b.o), pi=acosl(-1.L);
74     if (d>a.r+b.r) return 0;
75     if (d<=fabsl(a.r-b.r)) return pi*min(a.r, b.r)*min(a.r, b.r);
76     R x=acosl(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r), R(-1), R(1)));
77     R y=acosl(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r), R(-1), R(1)));
78     return a.r*a.r*(x-sinl(2*x)/2)+b.r*b.r*(y-sinl(2*y)/2);
79 }
80 };

```

第十章 优化与可持久化

10.1 Max-plus 矩阵幂

$a[i][j]$ 为转移收益，不存在的边为 neg ； $power(k)$ 求恰好 k 步的最优转移，单位矩阵对角为 0。所有有限中间值绝对值小于 2^{119} 。 $O(n^3 \log k)$ ；2025 成都 B 的转移类型，具体题目的建模仍需调用者完成。

```
1 struct MaxPlusMatrix
2 {
3     using I=__int128_t;
4     static constexpr I neg=-(I(1)<<120);
5     int n; vector<vector<I>> a;
6     MaxPlusMatrix(int n,bool identity=false):n(n),a(n,vector<I>(n,neg))
7     { if (identity) for (int i=0;i<n;i++) a[i][i]=0; }
8     MaxPlusMatrix operator*(const MaxPlusMatrix &b) const
9     {
10         assert(n==b.n); MaxPlusMatrix c(n);
11         for (int i=0;i<n;i++) for (int k=0;k<n;k++) if (a[i][k]!=neg)
12             for (int j=0;j<n;j++) if (b.a[k][j]!=neg)
13                 c.a[i][j]=max(c.a[i][j],a[i][k]+b.a[k][j]);
14         return c;
15     }
16     MaxPlusMatrix power(unsigned long long e) const
17     {
18         MaxPlusMatrix b=*this,r(n,true);
19         for (;e>>=1,b=b*b) if (e&1) r=r*b;
20         return r;
21     }
22 };
```

10.2 离散李超树与最优直线编号

构造传入所有可能查询的横坐标， $\text{add}(\{k,b,\text{id}\})$ 插入直线， $\text{query}(x)$ 返回最小值与最小编号，要求 x 在预登记坐标内且至少有一条线。整数求值用 int128 ， k 、 b 、 x 为有符号 64 位且中间值可表示。单次 $O(\log n)$ 。

```

1 struct LiChao
2 {
3     using ll=long long; using I=__int128_t;
4     struct Line
5     {
6         ll k=0,b=0; int id=-1;
7         I value(ll x) const { return I(k)*x+b; }
8     };
9     struct Node { Line line; int left=-1,right=-1; };
10    vector<ll> xs; vector<Node> a{Node{}};
11    LiChao(vector<ll> x):xs(x)
12    { sort(xs.begin(),xs.end()); xs.erase(unique(xs.begin(),xs.end()),xs.end()); assert(!xs.empty()); }
13    bool better(Line u,Line v,ll x) const
14    { return v.id!=-1||(u.id!=-1&&pair{u.value(x),u.id}<pair{v.value(x),v.id}); }
15    void add(Line line) { assert(line.id>=0); add(0,0,(int)xs.size()-1,line); }
16    void add(int p,int l,int r,Line line)
17    {
18        int m=(l+r)/2;
19        if (better(line,a[p].line,xs[m])) swap(line,a[p].line);
20        if (l==r||line.id==-1) return;
21        bool left=better(line,a[p].line,xs[l]);
22        bool right=better(line,a[p].line,xs[r]);
23        if (!left&&!right) return;
24        int next=left?a[p].left:a[p].right;
25        if (next==-1)
26        {
27            next=(int)a.size(); a.push_back(Node{});
28            if (left) a[p].left=next; else a[p].right=next;
29        }
30        if (left) add(next,l,m,line); else add(next,m+1,r,line);
31    }
32    pair<I,int> query(ll x) const
33    {
34        int idx=lower_bound(xs.begin(),xs.end(),x)-xs.begin();
35        assert(idx<(int)xs.size()&&xs[idx]==x);
36        int p=0,l=0,r=(int)xs.size()-1; Line best;
37        while (p!=-1)
38        {
39            if (better(a[p].line,best,x)) best=a[p].line;
40            if (l==r) break;
41            int m=(l+r)/2;
42            if (xs[idx]<=m) { p=a[p].left; r=m; } else { p=a[p].right; l=m+1; }
43        }
44        assert(best.id!=-1); return {best.value(x),best.id};
45    }
46 };

```

10.3 主席树：静态区间第 k 小

构造传入 0-based 数组， $kth(l,r,k)$ 的区间使用 1-based 闭区间；要求 $1 \leq k \leq r-l+1$ 。离散化后每个前缀一个版本，单次 $O(\log n)$ ，总空间 $O(n \log n)$ 。

```

1 struct PersistentKth
2 {
3     struct Node { int l=0,r=0,sum=0; };
4     vector<Node> t{Node{}}; vector<int> root{0}; vector<long long> vals;
5     PersistentKth(const vector<long long> &a):vals(a)
6     {
7         sort(vals.begin(),vals.end()); vals.erase(unique(vals.begin(),vals.end()),vals.end());
8         for (long long x:a)
9             { int p=lower_bound(vals.begin(),vals.end(),x)-vals.begin(); root.push_back(add(root.back(),0,(int)vals.size()-1,p)); }
10    }
11    int add(int old,int l,int r,int x)
12    {
13        int p=t.size(); Node copy=t[old]; t.push_back(copy); ++t[p].sum;
14        if (l<r)
15            {
16                int m=(l+r)/2;
17                if (x<=m) { int v=add(t[old].l,l,m,x); t[p].l=v; }
18                else { int v=add(t[old].r,m+1,r,x); t[p].r=v; }
19            }
20        return p;
21    }
22    long long kth(int l,int r,int k) const
23    {
24        assert(1<=l&&l<=r&&r<(int)root.size()&&1<=k&&k<=r-l+1);
25        int u=root[l-1],v=root[r],L=0,R=(int)vals.size()-1;
26        while (L<R)
27            {
28                int m=(L+R)/2,c=t[t[v].l].sum-t[t[u].l].sum;
29                if (k<=c) { u=t[u].l; v=t[v].l; R=m; }
30                else { k-=c; u=t[u].r; v=t[v].r; L=m+1; }
31            }
32        return vals[L];
33    }
34 };

```

第十一章 图论进阶

11.1 矩形二分图最小权匹配

`solve(cost)` 输入 $n \times m$ 完整权值矩阵, $0 \leq n \leq m$; 每行匹配不同列, 返回最小费用及列号 (0-based)。允许负费用, 使用 int128 势能; $O(n^2m)$ 。禁止边不能用未经证明的有限大常数随意替代, 当前接口仅表示完整二分图。

```
1 struct Hungarian
2 {
3     using I=__int128_t;
4     // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
5     static pair<I,vector<int>> solve(const vector<vector<long long>> &cost)
6     {
7         int n=cost.size(); if (!n) return {0,{}};
8         int m=cost[0].size(); assert(n<=m);
9         vector<I> u(n+1),v(m+1); vector<int> p(m+1),way(m+1);
10        const I inf=I(1)<<120;
11        for (int i=1;i<=n;i++)
12        {
13            assert((int)cost[i-1].size()==m); p[0]=i; int j0=0;
14            vector<I> dis(m+1,inf); vector<bool> used(m+1);
15            do
16            {
17                used[j0]=true; int i0=p[j0],j1=0; I delta=inf;
18                for (int j=1;j<=m;j++) if (!used[j])
19                {
20                    I cur=I(cost[i0-1][j-1])-u[i0]-v[j];
21                    if (cur<dis[j]) { dis[j]=cur; way[j]=j0; }
22                    if (dis[j]<delta) { delta=dis[j]; j1=j; }
23                }
24                for (int j=0;j<=m;j++)
25                    if (used[j]) { u[p[j]]+=delta; v[j]-=delta; }
26                else dis[j]-=delta;
27                j0=j1;
28            } while (p[j0]);
29            do { int j1=way[j0]; p[j0]=p[j1]; j0=j1; } while (j0);
30        }
31        vector<int> ans(n); for (int j=1;j<=m;j++) if (p[j]) ans[p[j]-1]=j-1;
32        return {-v[0],ans};
33    }
34};
```

11.2 有向最小树形图

`solve(n,root,edges)`, 点号 $0..n-1$, $n \geq 1$; 返回以 `root` 为根向外可达的最小树形图费用, 无解 `nullopt`。允许负权、重边、自环 (自环不选); $O(VE)$ 。当前仅返回费用, 不恢复边方案。所有中间值须远小于 2^{120} 。

```

1 struct Arborescence
2 {
3     using I=__int128_t;
4     struct Edge { int u,v; I w; };
5     // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
6     static optional<I> solve(int n,int root,vector<Edge> e)
7     {
8         const I inf=I(1)<<120; I ans=0;
9         while (true)
10         {
11             vector<I> in(n,inf); vector<int> pre(n,-1),id(n,-1),vis(n,-1);
12             for (auto a:e) if (a.u!=a.v&&a.w<in[a.v]) { in[a.v]=a.w; pre[a.v]=a.u; }
13             in[root]=0;
14             for (I x:in) { if (x==inf) return nullopt; ans+=x; }
15             int cnt=0;
16             for (int i=0;i<n;i++)
17             {
18                 int u=i;
19                 while (vis[u]!=i&&id[u]==-1&&u!=root) { vis[u]=i; u=pre[u]; }
20                 if (u!=root&&id[u]==-1)
21                 {
22                     id[u]=cnt;
23                     for (int v=pre[u];v!=u;v=pre[v]) id[v]=cnt;
24                     ++cnt;
25                 }
26             }
27             if (!cnt) return ans;
28             for (int i=0;i<n;i++) if (id[i]==-1) id[i]=cnt++;
29             for (auto &a:e)
30             { int v=a.v; a.u=id[a.u]; a.v=id[a.v]; if (a.u!=a.v) a.w-=in[v]; }
31             root=id[root]; n=cnt;
32         }
33     }
34 };

```

11.3 无向全局最小割

$\text{solve}(w)$, $n \geq 2$; w 是对称非负权矩阵, 重边需先累加, 主对角置零。返回割权与一侧顶点编号 (0-based), 允许不连通图。 $O(n^3)$, 总权值小于 2^{120} 。

```

1 struct StoerWagner
2 {
3     using I=__int128_t;
4     // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
5     static pair<I,vector<int>> solve(vector<vector<I>> w)
6     {
7         int n=w.size(); assert(n>=2); vector<int> active(n); iota(active.begin(),active.end(),0);
8         vector<vector<int>> group(n); for (int i=0;i<n;i++) group[i]={i};
9         I best=I(1)<<120; vector<int> side;
10        while (active.size()>1)
11        {
12            vector<I> dis(n); vector<bool> used(n); int prev=-1;
13            for (int step=0;step<(int)active.size();step++)
14            {
15                int sel=-1;
16                for (int v:active) if (!used[v]&&(sel==-1||dis[v]>dis[sel])) sel=v;
17                if (step+1==(int)active.size())
18                {
19                    if (dis[sel]<best) { best=dis[sel]; side=group[sel]; }
20                    for (int v:active) if (v!=prev&&v!=sel) w[prev][v]=w[v][prev]=w[prev][v]+w[sel][v];
21                    group[prev].insert(group[prev].end(),group[sel].begin(),group[sel].end());
22                    active.erase(find(active.begin(),active.end(),sel)); break;
23                }
24                used[sel]=true; prev=sel;
25                for (int v:active) if (!used[v]) dis[v]+=w[sel][v];
26            }
27        }
28        return {best,side};
29    }
30 };

```

第十二章 精确几何进阶

12.1 整数最近点对与 Minkowski 和

`closest_pair(points)` 返回最短距离平方，少于两个点为 `nullopt`，重复点返回 0， $O(n \log n)$ ，全程精确整数。
`minkowski` 输入 strict CCW 凸包，返回不重复首点的凸包；非退化 $O(n+m)$ ，退化时 $O((n+m)\log(n+m))$ 。坐标及坐标和须满足整数几何范围。

```
1 struct GeometryExtra
2 {
3     using G=IntegerGeometry; using P=G::Point; using I=G::I;
4     // No pair exists for n<2. Exact squared distance,  $O(n \log n)$ .
5     static optional<I> closest_pair(vector<P> p)
6     {
7         int n=p.size(); if (n<2) return nullopt;
8         sort(p.begin(),p.end());
9         for (int i=1;i<n;i++) if (p[i]==p[i-1]) return I(0);
10        vector<P> tmp(n);
11        auto by_y=[](P a,P b) { return tie(a.y,a.x)<tie(b.y,b.x); };
12        function<I(int,int)> solve=[&](int l,int r)->I
13        {
14            if (r-l<=3)
15            {
16                I best=I(1)<<120;
17                for (int i=l;i<r;i++) for (int j=i+1;j<r;j++) best=min(best,G::dist2(p[i],p[j]));
18                sort(p.begin()+l,p.begin()+r,by_y); return best;
19            }
20            int m=(l+r)/2; long long x=p[m].x;
21            I best=min(solve(l,m),solve(m,r));
22            merge(p.begin()+l,p.begin()+m,p.begin()+m,p.begin()+r,tmp.begin()+l,by_y);
23            copy(tmp.begin()+l,tmp.begin()+r,p.begin()+l);
24            vector<P> strip;
25            for (int i=l;i<r;i++) if ((I(p[i].x)-x)*(I(p[i].x)-x)<best)
26            {
27                for (int j=(int)strip.size()-1;j>=0;j--)
28                {
29                    I dy=I(p[i].y)-strip[j].y; if (dy*dy>=best) break;
30                    best=min(best,G::dist2(p[i],strip[j]));
31                }
32                strip.push_back(p[i]);
33            }
34            return best;
35        };
36        return solve(0,n);
37    }
38    // Inputs strict CCW hulls. Coordinate sums must obey IntegerGeometry's bound.
39    static vector<P> minkowski(vector<P> a,vector<P> b)
40    {
41        if (a.empty()||b.empty()) return {};
42        auto plus=[](P u,P v) { return P{u.x+v.x,u.y+v.y}; };
43        if (a.size()<3||b.size()<3)
44        { vector<P> p; for (P u:a) for (P v:b) p.push_back(plus(u,v)); return G::hull(p); }
45        auto start=[](vector<P> &p)
46        {
47            auto it=min_element(p.begin(),p.end(),[](P u,P v) { return tie(u.y,u.x)<tie(v.y,v.x); });
48            rotate(p.begin(),it,p.end());
49        };
50        start(a); start(b); int n=a.size(),m=b.size(),i=0,j=0; vector<P> result;
51        while (i<n||j<m)
52        {
53            result.push_back(plus(a[i%n],b[j%m]));
54            if (i==n) { ++j; continue; }
55            if (j==m) { ++i; continue; }
56            P u{a[(i+1)%n].x-a[i].x,a[(i+1)%n].y-a[i].y};
```

```

57         P v{b[(j+1)%m].x-b[j].x,b[(j+1)%m].y-b[j].y};
58         I c=G::cross({0,0},u,v);
59         if (c>=0) ++i;
60         if (c<=0) ++j;
61     }
62     return result;
63 }
64 };

```

12.2 整数三维共线、共面与方向

坐标绝对值 $\leq 10^9$ ；orient(a,b,c,d) 为有向六倍四面体体积，0 表示共面；collinear 判三点共线，on_segment 支持退化线段。int128 精确判定，不包含三维凸包构造。

```

1 struct IntegerGeometry3D
2 {
3     using I=__int128_t;
4     // |coordinates|<=1e9: triple products fit signed int128.
5     struct Point { long long x,y,z; };
6     struct Vector { I x,y,z; };
7     static Vector diff(Point a,Point b) { return {I(a.x)-b.x,I(a.y)-b.y,I(a.z)-b.z}; }
8     static Vector cross(Vector a,Vector b)
9     { return {a.y*b.z-a.z*b.y,a.z*b.x-a.x*b.z,a.x*b.y-a.y*b.x}; }
10    static I dot(Vector a,Vector b) { return a.x*b.x+a.y*b.y+a.z*b.z; }
11    static I orient(Point a,Point b,Point c,Point d)
12    { return dot(cross(diff(b,a),diff(c,a)),diff(d,a)); }
13    static bool collinear(Point a,Point b,Point c)
14    { auto v=cross(diff(b,a),diff(c,a)); return !v.x&&!v.y&&!v.z; }
15    static bool on_segment(Point p,Point a,Point b)
16    { return collinear(p,a,b)&&dot(diff(a,p),diff(b,p))<=0; }
17 };

```

第二部分

传统接口风格

第十三章 数据结构

13.1 并查集

算法约束见紧凑版本第 2 页。本节代码独立可抄；采用下列实际接口名。

先声明容量模板参数，再调用 Init(n) 初始化；大对象必须置于全局或 static 存储。

```
1 template<int N> struct Disjoint_Set
2 {
3     int n, fa[N + 1], siz[N + 1];
4     void Init ( int size )
5     {
6         assert(size <= N);
7         n = size;
8         iota(fa, fa + n + 1, 0);
9         fill(siz, siz + n + 1, 1);
10    }
11    int Find ( int x )
12    {
13        int root = x;
14        while ( root != fa[root] )
15            root = fa[root];
16        while ( x != root )
17        {
18            int next = fa[x];
19            fa[x] = root;
20            x = next;
21        }
22        return root;
23    }
24    bool Merge ( int x, int y )
25    {
26        x = Find(x); y = Find(y);
27        if ( x == y ) return false;
28        if ( siz[x] < siz[y] ) swap(x, y);
29        fa[y] = x; siz[x] += siz[y];
30        return true;
31    }
32 };
```

13.2 可撤销并查集

算法约束见紧凑版本第 2 页。本节代码独立可抄；采用下列实际接口名。

```
1 template<int N> struct Rollback_DSU
2 {
3     int fa[N + 1], siz[N + 1];
4     vector<pair<int,int>> history;
5     void Init ( int n )
6     {
7         assert(n <= N); history.clear();
8         iota(fa, fa + n + 1, 0); fill(siz, siz + n + 1, 1);
9     }
10    int Find ( int x ) const
11    {
12        while ( fa[x] != x ) x = fa[x];
13        return x;
14    }
15    int Snapshot () const { return (int)history.size(); }
16    bool Merge ( int x, int y )
17    {
18        x = Find(x); y = Find(y);
19        if ( x == y ) return false;
20        if ( siz[x] < siz[y] ) swap(x, y);
21        history.push_back({y, siz[x]});
22        fa[y] = x; siz[x] += siz[y];
23        return true;
24    }
25    void Rollback ( int time )
26    {
27        assert(0 <= time && time <= Snapshot());
28        while ( Snapshot() > time )
29        {
30            auto [y, size] = history.back(); history.pop_back();
31            siz[fa[y]] = size; fa[y] = y;
32        }
33    }
34};
```

13.3 树状数组与第 k 小

算法约束见紧凑版本第 3 页。本节代码独立可抄；采用下列实际接口名。

先声明容量模板参数，再调用 Init(n) 初始化；大对象必须置于全局或 static 存储。

```
1 template<int N, class T = long long> struct Binary_Indexed_Tree
2 {
3     int n;
4     T tree[N + 1];
5     void Init ( int size )
6     {
7         assert(size <= N); n = size;
8         fill(tree, tree + n + 1, T(0));
9     }
10    void Insert ( int x, T value )
11    {
12        assert(x >= 1);
13        for ( ; x <= n; x += x & -x ) tree[x] += value;
14    }
15    T Sum ( int x ) const
16    {
17        T answer = 0;
18        for ( ; x; x -= x & -x ) answer += tree[x];
19        return answer;
20    }
21    T Query ( int L, int R ) const { return Sum(R) - Sum(L - 1); }
22    int Kth ( T k ) const
23    {
24        assert(k > 0); int now = 0, step = 1;
25        while ( step <= n / 2 ) step <<= 1;
26        for ( ; step; step >>= 1 )
27            if ( now + step <= n && tree[now + step] < k )
28            {
29                now += step; k -= tree[now];
30            }
31        return now + 1;
32    }
33 };
```

13.4 区间加与区间和

算法约束见紧凑版本第 3 页。本节代码独立可抄；采用下列实际接口名。

先声明容量模板参数，再调用 Init(n) 初始化；大对象必须置于全局或 static 存储。

```

1  template<int N> struct Segment_Tree
2  {
3      int n;
4      long long tree[4 * N + 5], tag[4 * N + 5];
5      void Init ( int size )
6      {
7          assert(size > 0 && size <= N); n = size;
8          fill(tree, tree + 4 * n + 5, 0);
9          fill(tag, tag + 4 * n + 5, 0);
10     }
11     void Update ( int now, int L, int R, long long value )
12     {
13         tree[now] += (R - L + 1) * value;
14         tag[now] += value;
15     }
16     void Push_Up ( int now ) { tree[now] = tree[now << 1] + tree[now << 1 | 1]; }
17     void Push_Down ( int now, int L, int R )
18     {
19         int mid = (L + R) >> 1;
20         Update(now << 1, L, mid, tag[now]);
21         Update(now << 1 | 1, mid + 1, R, tag[now]);
22         tag[now] = 0;
23     }
24     void Insert ( int ql, int qr, long long value ) { Insert(1, 1, n, ql, qr, value); }
25     void Insert ( int now, int L, int R, int ql, int qr, long long value )
26     {
27         if ( ql <= L && R <= qr ) return Update(now, L, R, value);
28         Push_Down(now, L, R);
29         int mid = (L + R) >> 1;
30         if ( ql <= mid ) Insert(now << 1, L, mid, ql, qr, value);
31         if ( qr > mid ) Insert(now << 1 | 1, mid + 1, R, ql, qr, value);
32         Push_Up(now);
33     }
34     long long Query ( int ql, int qr ) { return Query(1, 1, n, ql, qr); }
35     long long Query ( int now, int L, int R, int ql, int qr )
36     {
37         if ( ql <= L && R <= qr ) return tree[now];
38         Push_Down(now, L, R);
39         int mid = (L + R) >> 1;
40         long long answer = 0;
41         if ( ql <= mid ) answer += Query(now << 1, L, mid, ql, qr);
42         if ( qr > mid ) answer += Query(now << 1 | 1, mid + 1, R, ql, qr);
43         return answer;
44     }
45 };

```

13.5 64 位异或线性基

算法约束见紧凑版本第 4 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Xor_Basis
2 {
3     unsigned long long base[64]{};
4     bool Insert ( unsigned long long x )
5     {
6         for ( int i = 63; i >= 0; i -- )
7         {
8             if ( !(x >> i & 1) ) continue;
9             if ( !base[i] ) { base[i] = x; return true; }
10            x ^= base[i];
11        }
12        return false;
13    }
14    unsigned long long Query ( unsigned long long x = 0 ) const
15    {
16        for ( int i = 63; i >= 0; i -- ) x = max(x, x ^ base[i]);
17        return x;
18    }
19 };
```

第十四章 网络流

14.1 最大流与方案

算法约束见紧凑版本第 5 页。本节代码独立可抄；采用下列实际接口名。

先声明容量模板参数，再调用 Init(n) 初始化；大对象必须置于全局或 static 存储。

```
1 template<int N, int M> struct Network_Flow
2 {
3     using ll = long long;
4     struct Edge { int to, next; ll cap, initial; } edge[2 * M];
5     int n, tot, head[N + 1], deep[N + 1], cur[N + 1], que[N + 1];
6     void Init ( int size )
7     {
8         assert(size <= N); n = size; tot = 0;
9         fill(head, head + n + 1, -1);
10    }
11    int Add_Edge ( int u, int v, ll cap )
12    {
13        assert(cap >= 0 && tot + 2 <= 2 * M);
14        int id = tot;
15        edge[tot] = {v, head[u], cap, cap}; head[u] = tot ++;
16        edge[tot] = {u, head[v], 0, 0}; head[v] = tot ++;
17        return id;
18    }
19    bool Bfs ( int s, int t )
20    {
21        fill(deep, deep + n + 1, -1);
22        int L = 0, R = 0; que[R ++] = s; deep[s] = 0;
23        while ( L < R )
24        {
25            int now = que[L ++];
26            for ( int i = head[now]; i != -1; i = edge[i].next )
27            {
28                int to = edge[i].to;
29                if ( edge[i].cap && deep[to] == -1 )
30                { deep[to] = deep[now] + 1; que[R ++] = to; }
31            }
32        }
33        return deep[t] != -1;
34    }
35    ll Dfs ( int now, int t, ll flow )
36    {
37        if ( now == t ) return flow;
38        for ( int &i = cur[now]; i != -1; i = edge[i].next )
39        {
40            int to = edge[i].to;
41            if ( !edge[i].cap || deep[to] != deep[now] + 1 ) continue;
42            ll delta = Dfs(to, t, min(flow, edge[i].cap));
43            if ( delta )
44            { edge[i].cap -= delta; edge[i ^ 1].cap += delta; return delta; }
45        }
46        return 0;
47    }
48    ll Max_Flow ( int s, int t, ll limit = LLONG_MAX )
49    {
50        assert(s != t && limit >= 0); ll answer = 0;
51        while ( answer < limit && Bfs(s, t) )
52        {
53            copy(head, head + n + 1, cur);
54            while ( answer < limit )
55            {
56                ll delta = Dfs(s, t, limit - answer);
57                if ( !delta ) break;
58                answer += delta;
```

```

59     }
60 }
61 return answer;
62 }
63 ll Used ( int id ) const { return edge[id].initial - edge[id].cap; }
64 vector<int> Cut ( int s )
65 {
66     Bfs(s, s); vector<int> answer;
67     for ( int i = 1; i <= n; i ++ ) if ( deep[i] != -1 ) answer.push_back(i);
68     return answer;
69 }
70 };

```

14.2 势能最小费用最大流

算法约束见紧凑版本第 6 页。本节代码独立可抄；采用下列实际接口名。
使用 Insert 加边、Flow 求流、Used 查看方案。

```

1 struct Min_Cost_Flow
2 {
3     using ll = long long;
4     using i128 = __int128_t;
5     struct Edge
6     {
7         int from, to;
8         ll cap, cost, initial;
9     };
10    int n;
11    vector<Edge> e;
12    vector<vector<int>> g;
13    Min_Cost_Flow(int n) : n(n), g(n + 1) {}
14    int Insert(int u, int v, ll cap, ll cost)
15    {
16        assert(cap >= 0 && cost != LLONG_MIN);
17        int id = (int)e.size();
18        e.push_back({u, v, cap, cost, cap});
19        e.push_back({v, u, 0, -cost, 0});
20        g[u].push_back(id);
21        g[v].push_back(id + 1);
22        return id;
23    }
24    ll Used(int id) const
25    {
26        return e[id].initial - e[id].cap;
27    }
28    // Negative costs allowed; initial residual graph must have no negative cycle.
29    pair<ll, i128> Flow(int s, int t, ll limit = LLONG_MAX)
30    {
31        assert(s != t && limit >= 0);
32        const i128 inf = i128(1) << 120;
33        vector<i128> h(n + 1, 0), d(n + 1);
34        vector<int> pre(n + 1);
35        // Super-source Bellman-Ford: feasible potentials for every vertex.
36        for ( int it = 0; it < n; it++ )
37        {
38            bool changed = false;
39            for ( auto &a : e )
40                if ( a.cap && h[a.to] > h[a.from] + a.cost )
41                {
42                    h[a.to] = h[a.from] + a.cost;
43                    changed = true;
44                }
45            if ( !changed )
46                break;
47            if ( it == n - 1 )
48                throw invalid_argument("negative cycle");
49        }
50        ll f = 0;

```

```

51     i128 cost = 0;
52     while ( f < limit )
53     {
54         fill(d.begin(), d.end(), inf);
55         d[s] = 0;
56         priority_queue<pair<i128, int>, vector<pair<i128, int>>,
57                     greater<pair<i128, int>>>
58             q;
59         q.push({0, s});
60         while ( !q.empty() )
61         {
62             auto [du, u] = q.top();
63             q.pop();
64             if ( du != d[u] )
65                 continue;
66             for ( int id : g[u] )
67             {
68                 auto &a = e[id];
69                 i128 nd = du + a.cost + h[u] - h[a.to];
70                 if ( a.cap && nd < d[a.to] )
71                 {
72                     d[a.to] = nd;
73                     pre[a.to] = id;
74                     q.push({nd, a.to});
75                 }
76             }
77         }
78         if ( d[t] == inf )
79             break;
80         for ( int u = 1; u <= n; u++ )
81             if ( d[u] != inf )
82                 h[u] += d[u];
83         ll take = limit - f;
84         for ( int u = t; u != s; u = e[pre[u]].from )
85             take = min(take, e[pre[u]].cap);
86         for ( int u = t; u != s; u = e[pre[u]].from )
87         {
88             int id = pre[u];
89             e[id].cap -= take;
90             e[id ^ 1].cap += take;
91             cost += i128(take) * e[id].cost;
92         }
93         f += take;
94     }
95     return {f, cost};
96 }
97 };

```

14.3 上下界可行循环流

算法约束见紧凑版本第 7 页。本节代码独立可抄；采用下列实际接口名。

```

1 template<int N, int M> struct Bounded_Circulation
2 {
3     int n, tot;
4     Network_Flow<N + 2, M + N> graph;
5     long long balance[N + 1], lower[M];
6     int id[M];
7     bool solved, feasible;
8     void Init ( int size )
9     {
10         assert(size <= N); n = size; tot = 0;
11         solved = feasible = false; graph.Init(n + 2);
12         fill(balance, balance + n + 1, 0);
13     }
14     int Insert ( int u, int v, long long L, long long R )
15     {
16         assert(!solved && 0 <= L && L <= R && tot < M);
17         id[tot] = graph.Add_Edge(u, v, R - L); lower[tot] = L;
18         balance[u] -= L; balance[v] += L;
19         return tot++;
20     }
21     bool Solve ()
22     {
23         assert(!solved); solved = true;
24         long long need = 0;
25         for ( int u = 1; u <= n; u++ )
26         {
27             if ( balance[u] > 0 )
28                 { graph.Add_Edge(n + 1, u, balance[u]); need += balance[u]; }
29             else if ( balance[u] < 0 ) graph.Add_Edge(u, n + 2, -balance[u]);
30         }
31         return feasible = (graph.Max_Flow(n + 1, n + 2) == need);
32     }
33     long long Used ( int index ) const
34     {
35         assert(feasible);
36         return lower[index] + graph.Used(id[index]);
37     }
38 };

```

第十五章 图论

15.1 非负权最短路

算法约束见紧凑版本第 8 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Shortest_Path
2 {
3     using ll = long long;
4     static constexpr ll inf = LLONG_MAX;
5     int n;
6     vector<vector<pair<int, ll>>> g;
7     vector<ll> dis;
8     vector<int> pre;
9     Shortest_Path(int n) : n(n), g(n + 1) {}
10    void Insert(int u, int v, ll w)
11    {
12        assert(w >= 0);
13        g[u].push_back({v, w});
14    }
15    void Run(int s)
16    {
17        dis.assign(n + 1, inf);
18        pre.assign(n + 1, -1);
19        dis[s] = 0;
20        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
21        q.push({0, s});
22        while (!q.empty())
23        {
24            auto [d, u] = q.top();
25            q.pop();
26            if (d != dis[u])
27                continue;
28            for (auto [v, w] : g[u])
29                if (w < inf - d && d + w < dis[v])
30                {
31                    dis[v] = d + w;
32                    pre[v] = u;
33                    q.push({dis[v], v});
34                }
35        }
36    }
37    vector<int> Path(int t) const
38    {
39        if (dis[t] == inf)
40            return {};
41        vector<int> ans;
42        for (; t != -1; t = pre[t])
43            ans.push_back(t);
44        reverse(ans.begin(), ans.end());
45        return ans;
46    }
47 };
```

15.2 强连通分量

算法约束见紧凑版本第 9 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Strong_Component
2 {
3     int n, cnt = 0;
4     vector<vector<int>> g, rg;
5     vector<int> bel;
```

```

6 Strong_Component(int n) : n(n), g(n + 1), rg(n + 1) {}
7 void Insert(int u, int v)
8 {
9     g[u].push_back(v);
10    rg[v].push_back(u);
11 }
12 // Iterative Kosaraju; component IDs are in topological order, 1..cnt.
13 void Run()
14 {
15     vector<int> vis(n + 1), order;
16     cnt = 0;
17     bel.assign(n + 1, 0);
18     for ( int s = 1; s <= n; s++ )
19         if ( !vis[s] )
20         {
21             vector<pair<int, int>> st{{s, 0}};
22             vis[s] = 1;
23             while ( !st.empty() )
24             {
25                 int u = st.back().first, &i = st.back().second;
26                 if ( i == (int)g[u].size() )
27                 {
28                     order.push_back(u);
29                     st.pop_back();
30                 }
31                 else
32                 {
33                     int v = g[u][i++];
34                     if ( !vis[v] )
35                     {
36                         vis[v] = 1;
37                         st.push_back({v, 0});
38                     }
39                 }
40             }
41         }
42     reverse(order.begin(), order.end());
43     for ( int s : order )
44         if ( !bel[s] )
45         {
46             ++cnt;
47             vector<int> st{s};
48             bel[s] = cnt;
49             while ( !st.empty() )
50             {
51                 int u = st.back();
52                 st.pop_back();
53                 for ( int v : rg[u] )
54                     if ( !bel[v] )
55                     {
56                         bel[v] = cnt;
57                         st.push_back(v);
58                     }
59             }
60         }
61 }
62 };

```

15.3 2-SAT 与任意可行方案

算法约束见紧凑版本第 9 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Two_SAT
2 {
3     int n;
4     Strong_Component g;
5     vector<int> ans;
6     Two_SAT(int n) : n(n), g(2 * n) {}
7     int id(int x, bool value) const
8     {
9         return 2 * x - 1 + value;
10    }
11    void Insert(int x, bool a, int y, bool b)
12    {
13        g.Insert(id(x, !a), id(y, b));
14        g.Insert(id(y, !b), id(x, a));
15    }
16    bool Solve()
17    {
18        g.Run();
19        ans.assign(n + 1, 0);
20        for (int x = 1; x <= n; x++)
21        {
22            if (g.bel[id(x, 0)] == g.bel[id(x, 1)])
23                return false;
24            ans[x] = g.bel[id(x, 1)] > g.bel[id(x, 0)];
25        }
26        return true;
27    }
28 };

```

15.4 二分图最大匹配

算法约束见紧凑版本第 10 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Bipartite_Matching
2 {
3     int n, m;
4     vector<vector<int>> g;
5     vector<int> l, r, dep;
6     Bipartite_Matching(int n, int m)
7         : n(n), m(m), g(n + 1), l(n + 1), r(m + 1), dep(n + 1)
8     {
9     }
10    void Insert(int u, int v)
11    {
12        g[u].push_back(v);
13    }
14    bool Bfs()
15    {
16        queue<int> q;
17        fill(dep.begin(), dep.end(), -1);
18        for (int u = 1; u <= n; u++)
19            if (!l[u])
20            {
21                dep[u] = 0;
22                q.push(u);
23            }
24        bool ok = false;
25        while (!q.empty())
26        {
27            int u = q.front();
28            q.pop();
29            for (int v : g[u])
30                if (!r[v])
31                    ok = true;

```

```
32         else if ( dep[r[v]] < 0 )
33         {
34             dep[r[v]] = dep[u] + 1;
35             q.push(r[v]);
36         }
37     }
38     return ok;
39 }
40 bool Dfs(int u)
41 {
42     for ( int v : g[u] )
43         if ( !r[v] || (dep[r[v]] == dep[u] + 1 && Dfs(r[v])) )
44         {
45             l[u] = v;
46             r[v] = u;
47             return true;
48         }
49     dep[u] = -1;
50     return false;
51 }
52 int Solve()
53 {
54     int ans = 0;
55     for ( int u = 1; u <= n; u++ )
56         ans += (l[u] != 0);
57     while ( Bfs() )
58         for ( int u = 1; u <= n; u++ )
59             if ( !l[u] )
60                 ans += Dfs(u);
61     return ans;
62 }
63 };
```

15.5 割点与桥

算法约束见紧凑版本第 11 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Low_Link
2 {
3     int n, timer = 0;
4     vector<vector<pair<int, int>>> g;
5     vector<int> dfn, low, cut, bridge;
6     int edges = 0;
7     Low_Link(int n) : n(n), g(n + 1) {}
8     int Insert(int u, int v)
9     {
10         g[u].push_back({v, edges});
11         g[v].push_back({u, edges});
12         return edges++;
13     }
14     void Dfs(int u, int pe)
15     {
16         dfn[u] = low[u] = ++timer;
17         int children = 0;
18         for ( auto [v, id] : g[u] )
19         {
20             if ( id == pe )
21                 continue;
22             if ( !dfn[v] )
23             {
24                 ++children;
25                 Dfs(v, id);
26                 low[u] = min(low[u], low[v]);
27                 if ( low[v] > dfn[u] )
28                     bridge[id] = 1;
29                 if ( pe != -1 && low[v] >= dfn[u] )
30                     cut[u] = 1;
31             }
32             else
33                 low[u] = min(low[u], dfn[v]);
34         }
35         if ( pe == -1 && children > 1 )
36             cut[u] = 1;
37     }
38     void Run()
39     {
40         timer = 0;
41         dfn.assign(n + 1, 0);
42         low = dfn;
43         cut = dfn;
44         bridge.assign(edges, 0);
45         for ( int u = 1; u <= n; u++ )
46             if ( !dfn[u] )
47                 Dfs(u, -1);
48     }
49 };

```

第十六章 树上算法

16.1 重链剖分与 LCA

算法约束见紧凑版本第 12 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Heavy_Light_Decomposition
2 {
3     int n, timer = 0;
4     vector<vector<int>> g;
5     vector<int> fa, dep, siz, son, top, dfn, rk;
6     Heavy_Light_Decomposition(int n)
7         : n(n), g(n + 1), fa(n + 1), dep(n + 1), siz(n + 1), son(n + 1), top(n + 1),
8           dfn(n + 1), rk(n + 1)
9     {
10    }
11    void Insert(int u, int v)
12    {
13        g[u].push_back(v);
14        g[v].push_back(u);
15    }
16    // Connected tree only; iterative build avoids chain-shaped DFS stack overflow.
17    void Build(int root = 1)
18    {
19        fill(fa.begin(), fa.end(), 0);
20        fill(son.begin(), son.end(), 0);
21        vector<int> order{root};
22        fa[root] = root;
23        dep[root] = 0;
24        for ( int i = 0; i < (int)order.size(); i++ )
25        {
26            int u = order[i];
27            for ( int v : g[u] )
28                if ( v != fa[u] )
29                {
30                    fa[v] = u;
31                    dep[v] = dep[u] + 1;
32                    order.push_back(v);
33                }
34        }
35        assert((int)order.size() == n);
36        for ( int i = n - 1; i >= 0; i-- )
37        {
38            int u = order[i];
39            siz[u] = 1;
40            for ( int v : g[u] )
41                if ( fa[v] == u && v != u )
42                {
43                    siz[u] += siz[v];
44                    if ( !son[u] || siz[v] > siz[son[u]] )
45                        son[u] = v;
46                }
47        }
48        timer = 0;
49        vector<pair<int, int>> st{{root, root}};
50        while ( !st.empty() )
51        {
52            auto [u, t] = st.back();
53            st.pop_back();
54            for ( ; u; u = son[u] )
55            {
56                top[u] = t;
57                dfn[u] = ++timer;
58                rk[timer] = u;
59                for ( int v : g[u] )
```

```

60         if ( fa[v] == u && v != u && v != son[u] )
61             st.push_back({v, v});
62     }
63 }
64
65 int Lca(int u, int v) const
66 {
67     while ( top[u] != top[v] )
68     {
69         if ( dep[top[u]] < dep[top[v]] )
70             swap(u, v);
71         u = fa[top[u]];
72     }
73     return dep[u] < dep[v] ? u : v;
74 }
75 // Commutative operations only; edge=true excludes the LCA's position.
76 template <class F> void Path(int u, int v, F work, bool edge = false) const
77 {
78     while ( top[u] != top[v] )
79     {
80         if ( dep[top[u]] < dep[top[v]] )
81             swap(u, v);
82         work(dfn[top[u]], dfn[u]);
83         u = fa[top[u]];
84     }
85     if ( dep[u] > dep[v] )
86         swap(u, v);
87     if ( dfn[u] + edge <= dfn[v] )
88         work(dfn[u] + edge, dfn[v]);
89 }
90 };

```

第十七章 字符串

17.1 KMP、Z、Manacher 与最小表示

算法约束见紧凑版本第 13 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct String_Algorithm
2 {
3     static vector<int> Prefix(const string &s)
4     {
5         int n = (int)s.size();
6         vector<int> p(n);
7         for ( int i = 1; i < n; i++ )
8         {
9             int j = p[i - 1];
10            while ( j && s[i] != s[j] )
11                j = p[j - 1];
12            p[i] = j + (s[i] == s[j]);
13        }
14        return p;
15    }
16    static vector<int> Match(const string &s, const string &t)
17    {
18        assert(!t.empty());
19        auto p = Prefix(t);
20        vector<int> ans;
21        for ( int i = 0, j = 0; i < (int)s.size(); i++ )
22        {
23            while ( j && s[i] != t[j] )
24                j = p[j - 1];
25            j += (s[i] == t[j]);
26            if ( j == (int)t.size() )
27            {
28                ans.push_back(i - j + 1);
29                j = p[j - 1];
30            }
31        }
32        return ans;
33    }
34    static vector<int> Z_Function(const string &s)
35    {
36        int n = (int)s.size();
37        vector<int> a(n);
38        if ( n )
39            a[0] = n;
40        for ( int i = 1, l = 0, r = 0; i < n; i++ )
41        {
42            if ( i < r )
43                a[i] = min(r - i, a[i - l]);
44            while ( i + a[i] < n && s[a[i]] == s[i + a[i]] )
45                ++a[i];
46            if ( i + a[i] > r )
47            {
48                l = i;
49                r = i + a[i];
50            }
51        }
52        return a;
53    }
54    // odd[i]: radius including center; even[i]: pairs centered before i.
55    static pair<vector<int>, vector<int>> Manacher(const string &s)
56    {
57        int n = (int)s.size();
58        vector<int> odd(n), even(n);
59        for ( int i = 0, l = 0, r = -1; i < n; i++ )
```

```

60     {
61         int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
62         while ( i - k >= 0 && i + k < n && s[i - k] == s[i + k] )
63             ++k;
64         odd[i] = k--;
65         if ( i + k > r )
66         {
67             l = i - k;
68             r = i + k;
69         }
70     }
71     for ( int i = 0, l = 0, r = -1; i < n; i++ )
72     {
73         int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
74         while ( i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k] )
75             ++k;
76         even[i] = k--;
77         if ( i + k > r )
78         {
79             l = i - k - 1;
80             r = i + k;
81         }
82     }
83     return {odd, even};
84 }
85 static int Rotation(const string &s)
86 {
87     int n = (int)s.size(), i = 0, j = 1, k = 0;
88     if ( !n )
89         return 0;
90     while ( i < n && j < n && k < n )
91     {
92         unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
93         if ( a == b )
94         {
95             ++k;
96             continue;
97         }
98         if ( a > b )
99         {
100             i += k + 1;
101             if ( i == j )
102                 ++i;
103         }
104         else
105         {
106             j += k + 1;
107             if ( i == j )
108                 ++j;
109         }
110         k = 0;
111     }
112     return min(i, j);
113 }
114 };

```

17.2 AC 自动机出现次数

算法约束见紧凑版本第 14 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct AC_Automaton
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int fail = 0;
7     };
8     vector<Node> a{Node{}};
9     vector<int> order;

```

```

10  bool built = false;
11  int Insert(const string &s)
12  {
13      assert(!built && !s.empty());
14      int u = 0;
15      for ( char ch : s )
16      {
17          int c = ch - 'a';
18          assert(0 <= c && c < 26);
19          if ( !a[u].go[c] )
20          {
21              int v = (int)a.size();
22              a[u].go[c] = v;
23              a.push_back(Node{});
24          }
25          u = a[u].go[c];
26      }
27      return u;
28  }
29  void Build()
30  {
31      assert(!built);
32      built = true;
33      queue<int> q;
34      for ( int v : a[0].go )
35          if ( v )
36              q.push(v);
37      while ( !q.empty() )
38      {
39          int u = q.front();
40          q.pop();
41          order.push_back(u);
42          for ( int c = 0; c < 26; c++ )
43          {
44              int v = a[u].go[c];
45              if ( v )
46              {
47                  a[v].fail = a[a[u].fail].go[c];
48                  q.push(v);
49              }
50              else
51                  a[u].go[c] = a[a[u].fail].go[c];
52          }
53      }
54  }
55  vector<long long> count(const string &s) const
56  {
57      assert(built);
58      vector<long long> ans(a.size());
59      int u = 0;
60      for ( char ch : s )
61      {
62          u = a[u].go[ch - 'a'];
63          ++ans[u];
64      }
65      for ( int i = (int)order.size() - 1; i >= 0; i-- )
66          ans[a[order[i]].fail] += ans[order[i]];
67      return ans;
68  }
69  };

```

17.3 倍增后缀数组与 LCP

算法约束见紧凑版本第 15 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Suffix_Array
2 {
3     vector<int> sa, rk, lcp;
4     Suffix_Array(const string &s)
5     {
6         int n = (int)s.size();
7         sa.resize(n);
8         rk.resize(n);
9         lcp.assign(n, 0);
10        for ( int i = 0; i < n; i++ )
11        {
12            sa[i] = i;
13            rk[i] = (unsigned char)s[i];
14        }
15        vector<int> tmp(n);
16        for ( int k = 1; k < n; k *= 2 )
17        {
18            auto key = [&](int i) { return pair{rk[i], i + k < n ? rk[i + k] : -1}; };
19            sort(sa.begin(), sa.end(), [&](int i, int j) { return key(i) < key(j); });
20            tmp[sa[0]] = 0;
21            for ( int i = 1; i < n; i++ )
22                tmp[sa[i]] = tmp[sa[i - 1]] + (key[sa[i]] != key[sa[i - 1]]);
23            rk = tmp;
24            if ( rk[sa.back()] == n - 1 )
25                break;
26        }
27        for ( int i = 0; i < n; i++ )
28            rk[sa[i]] = i;
29        for ( int i = 0, k = 0; i < n; i++ )
30        {
31            if ( !rk[i] )
32            {
33                k = 0;
34                continue;
35            }
36            int j = sa[rk[i] - 1];
37            while ( i + k < n && j + k < n && s[i + k] == s[j + k] )
38                ++k;
39            lcp[rk[i]] = k;
40            if ( k )
41                --k;
42        }
43    }
44 };

```

17.4 后缀自动机

算法约束见紧凑版本第 16 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Suffix_Automaton
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int link = -1, len = 0;
7         long long occ = 0;
8     };
9     vector<Node> a{Node{}};
10    int last = 0;
11    void Extend(int c)
12    {
13        assert(0 <= c && c < 26);
14        int cur = (int)a.size();
15        a.push_back(Node{});

```

```

16     a[cur].len = a[last].len + 1;
17     a[cur].occ = 1;
18     int p = last;
19     while ( p != -1 && !a[p].go[c] )
20     {
21         a[p].go[c] = cur;
22         p = a[p].link;
23     }
24     if ( p == -1 )
25         a[cur].link = 0;
26     else
27     {
28         int q = a[p].go[c];
29         if ( a[p].len + 1 == a[q].len )
30             a[cur].link = q;
31         else
32         {
33             int clone = (int)a.size();
34             Node copy = a[q];
35             a.push_back(copy);
36             a[clone].len = a[p].len + 1;
37             a[clone].occ = 0;
38             while ( p != -1 && a[p].go[c] == q )
39             {
40                 a[p].go[c] = clone;
41                 p = a[p].link;
42             }
43             a[q].link = a[cur].link = clone;
44         }
45     }
46     last = cur;
47 }
48 vector<long long> Counts() const
49 {
50     vector<int> bucket(a[last].len + 1), order(a.size());
51     vector<long long> ans(a.size());
52     for ( auto &v : a )
53         ++bucket[v.len];
54     partial_sum(bucket.begin(), bucket.end(), bucket.begin());
55     for ( int i = 0; i < (int)a.size(); i++ )
56     {
57         order[--bucket[a[i].len]] = i;
58         ans[i] = a[i].occ;
59     }
60     for ( int i = (int)order.size() - 1; i > 0; i-- )
61         ans[a[order[i]].link] += ans[order[i]];
62     return ans;
63 }
64 long long Distinct() const
65 {
66     long long ans = 0;
67     for ( int i = 1; i < (int)a.size(); i++ )
68         ans += a[i].len - a[a[i].link].len;
69     return ans;
70 }
71 };

```

第十八章 数论

18.1 模运算、素性、CRT 与类欧几里德

算法约束见紧凑版本第 17 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Number_Theory
2 {
3     using ll = long long;
4     using ull = unsigned long long;
5     using i128 = __int128_t;
6     using u128 = __uint128_t;
7     static ull Mul(ull a, ull b, ull m)
8     {
9         return u128(a) * b % m;
10    }
11    static ull Power(ull a, ull b, ull m)
12    {
13        assert(m);
14        ull r = 1 % m;
15        a %= m;
16        for ( ; b >>= 1, a = Mul(a, a, m) )
17            if ( b & 1 )
18                r = Mul(r, a, m);
19        return r;
20    }
21    static bool Prime(ull n)
22    {
23        if ( n < 2 )
24            return false;
25        for ( ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37} )
26            if ( n % p == 0 )
27                return n == p;
28        ull d = n - 1;
29        int s = 0;
30        while ( !(d & 1) )
31        {
32            d >>= 1;
33            ++s;
34        }
35        for ( ull a :
36            {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL} )
37        {
38            if ( a % n == 0 )
39                continue;
40            ull x = Power(a, d, n);
41            if ( x == 1 || x == n - 1 )
42                continue;
43            bool ok = false;
44            for ( int j = 1; j < s; j++ )
45            {
46                x = Mul(x, x, n);
47                if ( x == n - 1 )
48                {
49                    ok = true;
50                    break;
51                }
52            }
53            if ( !ok )
54                return false;
55        }
56        return true;
57    }
58    static i128 Exgcd(i128 a, i128 b, i128 &x, i128 &y)
59    {
```

```

60     if ( !b )
61     {
62         x = 1;
63         y = 0;
64         return a;
65     }
66     i128 d = Exgcd(b, a % b, y, x);
67     y -= a / b * x;
68     return d;
69 }
70 static ll Inverse(ll a, ll m)
71 {
72     assert(m > 0);
73     a %= m;
74     if ( a < 0 )
75         a += m;
76     i128 x, y;
77     if ( Exgcd(a, m, x, y) != 1 )
78         return -1;
79     return (x % m + m) % m;
80 }
81 // x = r (mod m); false means inconsistent. Throws if lcm exceeds int64.
82 static bool Crt(ll &r, ll &m, ll b, ll n)
83 {
84     assert(m > 0 && n > 0);
85     r %= m;
86     if ( r < 0 )
87         r += m;
88     b %= n;
89     if ( b < 0 )
90         b += n;
91     i128 x, y;
92     ll g = (ll)Exgcd(m, n, x, y);
93     i128 diff = i128(b) - r;
94     if ( diff % g )
95         return false;
96     i128 q = n / g, k = (diff / g * x % q + q) % q, mod = i128(m) * q;
97     if ( mod > LLONG_MAX )
98         throw overflow_error("CRT modulus");
99     r = (r + i128(m) * k) % mod;
100    m = (ll)mod;
101    return true;
102 }
103 // sum_{i=0}^{n-1} floor((a*i+b)/m), signed a,b; 0<=n<=1e9, 1<=m<=1e9.
104 static i128 Floor_Sum(ll n, ll m, ll a, ll b)
105 {
106     assert(n >= 0 && m > 0);
107     i128 ans = 0;
108     auto norm = [&](ll &v)
109     {
110         ll q = v / m;
111         if ( v % m < 0 )
112             --q;
113         v = (ll)(i128(v) - i128(q) * m);
114         return q;
115     };
116     ans += i128(norm(a)) * n * (n - 1) / 2;
117     ans += i128(norm(b)) * n;
118     while ( true )
119     {
120         ans += i128(n) * (n - 1) / 2 * (a / m);
121         a %= m;
122         ans += i128(n) * (b / m);
123         b %= m;
124         i128 y = i128(a) * n + b;
125         if ( y < m )
126             break;
127         n = (ll)(y / m);
128         b = (ll)(y % m);
129         swap(a, m);
130     }

```

```

131     return ans;
132 }
133 };

```

18.2 Pollard–Rho 因数分解

算法约束见紧凑版本第 18 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Pollard_Rho
2 {
3     using ull = unsigned long long;
4     using u128 = __uint128_t;
5     mt19937_64 rng;
6     Pollard_Rho(ull seed = 712367821) : rng(seed) {}
7     ull Rho(ull n)
8     {
9         if ( n % 2 == 0 )
10             return 2;
11         for ( ;; )
12         {
13             ull c = rng() % (n - 1) + 1, x = rng() % n, y = x, d = 1;
14             auto f = [&](ull v) { return (Number_Theory::Mul(v, v, n) + u128(c)) % n; };
15             // Retry bounded attempts; Las Vegas: only exact divisors returned.
16             for ( int i = 0; i < 200000 && d == 1; i++ )
17             {
18                 x = f(x);
19                 y = f(f(y));
20                 d = gcd(x > y ? x - y : y - x, n);
21             }
22             if ( 1 < d && d < n )
23                 return d;
24         }
25     }
26     void Split(ull n, vector<ull> &a)
27     {
28         if ( n == 1 )
29             return;
30         if ( Number_Theory::Prime(n) )
31         {
32             a.push_back(n);
33             return;
34         }
35         ull d = Rho(n);
36         Split(d, a);
37         Split(n / d, a);
38     }
39     vector<ull> Factor(ull n)
40     {
41         assert(n >= 1);
42         vector<ull> a;
43         Split(n, a);
44         sort(a.begin(), a.end());
45         return a;
46     }
47 };

```

18.3 线性筛、欧拉函数与 Möbius 函数

算法约束见紧凑版本第 19 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Linear_Sieve
2 {
3     vector<int> Prime, lp, phi, mu;
4     Linear_Sieve(int n) : lp(n + 1), phi(n + 1), mu(n + 1)
5     {
6         if ( n )
7             phi[1] = mu[1] = 1;
8         for ( int i = 2; i <= n; i++ )
9         {
10             if ( !lp[i] )
11             {
12                 lp[i] = i;
13                 Prime.push_back(i);
14                 phi[i] = i - 1;
15                 mu[i] = -1;
16             }
17             for ( int p : Prime )
18             {
19                 if ( p > n / i )
20                     break;
21                 int j = i * p;
22                 lp[j] = p;
23                 if ( i % p == 0 )
24                 {
25                     phi[j] = phi[i] * p;
26                     mu[j] = 0;
27                     break;
28                 }
29                 phi[j] = phi[i] * (p - 1);
30                 mu[j] = -mu[i];
31             }
32         }
33     }
34 };
```

18.4 静态模整数

算法约束见紧凑版本第 19 页。本节代码独立可抄；采用下列实际接口名。

```

1 template <int mod> struct Mod_Int
2 {
3     int v;
4     Mod_Int(long long x = 0) : v((x % mod + mod) % mod) {}
5     Mod_Int operator+(Mod_Int b) const
6     {
7         return Mod_Int((long long)v + b.v);
8     }
9     Mod_Int operator-(Mod_Int b) const
10    {
11        return Mod_Int((long long)v - b.v);
12    }
13    Mod_Int operator*(Mod_Int b) const
14    {
15        return Mod_Int(1LL * v * b.v);
16    }
17    Mod_Int Pow(long long e) const
18    {
19        assert(e >= 0);
20        Mod_Int a = *this, r = 1;
21        for ( ; e >= 1, a = a * a )
22            if ( e & 1 )
23                r = r * a;
24        return r;
25    }
26    Mod_Int Inv() const
27    {
28        assert(v);
29        return Pow(mod - 2);
30    } // prime modulus
31    Mod_Int operator/(Mod_Int b) const
32    {
33        return *this * b.Inv();
34    }
35 };

```

18.5 阶乘组合数

算法约束见紧凑版本第 19 页。本节代码独立可抄；采用下列实际接口名。

```

1 template <int mod> struct Combination
2 {
3     using Z = Mod_Int<mod>;
4     vector<Z> fac, ifac;
5     Combination(int n) : fac(n + 1), ifac(n + 1)
6     {
7         assert(0 <= n && n < mod);
8         fac[0] = 1;
9         for ( int i = 1; i <= n; i++ )
10            fac[i] = fac[i - 1] * i;
11        ifac[n] = fac[n].Inv();
12        for ( int i = n; i; i-- )
13            ifac[i - 1] = ifac[i] * i;
14    }
15    Z Choose(int n, int k) const
16    {
17        if ( k < 0 || k > n )
18            return 0;
19        assert(n < (int)fac.size());
20        return fac[n] * ifac[k] * ifac[n - k];
21    }
22 };

```

第十九章 多项式

19.1 NTT、形式幂级数、FWT 与递推

算法约束见紧凑版本第 20 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Polynomial
2 {
3     static constexpr int mod = 998244353;
4     using Z = Mod_Int<mod>;
5     using Poly = vector<Z>;
6     static void Ntt(Poly &a, bool invert = false)
7     {
8         int n = (int)a.size();
9         assert(n > 0 && (n & (n - 1)) == 0 && n <= (1 << 23));
10        for ( int i = 1, j = 0; i < n; i++ )
11        {
12            int bit = n >> 1;
13            for ( ; j & bit; bit >>= 1 )
14                j ^= bit;
15            j ^= bit;
16            if ( i < j )
17                swap(a[i], a[j]);
18        }
19        for ( int len = 2; len <= n; len *= 2 )
20        {
21            Z step = Z(3).Pow((mod - 1) / len);
22            if ( invert )
23                step = step.Inv();
24            for ( int i = 0; i < n; i += len )
25            {
26                Z w = 1;
27                for ( int j = 0; j < len / 2; j++, w = w * step )
28                {
29                    Z u = a[i + j], v = a[i + j + len / 2] * w;
30                    a[i + j] = u + v;
31                    a[i + j + len / 2] = u - v;
32                }
33            }
34        }
35        if ( invert )
36        {
37            Z Inv = Z(n).Inv();
38            for ( auto &x : a )
39                x = x * Inv;
40        }
41    }
42    static Poly Multiply(Poly a, Poly b)
43    {
44        if ( a.empty() || b.empty() )
45            return {};
46        int size = (int)a.size() + (int)b.size() - 1, n = 1;
47        assert(size <= (1 << 23));
48        while ( n < size )
49            n *= 2;
50        a.resize(n);
51        b.resize(n);
52        Ntt(a);
53        Ntt(b);
54        for ( int i = 0; i < n; i++ )
55            a[i] = a[i] * b[i];
56        Ntt(a, true);
57        a.resize(size);
58        return a;
59    }
```

```

60 static Poly Inverse(const Poly &a, int n)
61 {
62     assert(n > 0 && !a.empty() && a[0].v);
63     Poly b{a[0].Inv()};
64     while ( (int)b.size() < n )
65     {
66         int m = min(n, 2 * (int)b.size());
67         Poly f(a.begin(), a.begin() + min(m, (int)a.size()));
68         auto t = Multiply(f, b);
69         t.resize(m);
70         for ( auto &x : t )
71             x = Z(0) - x;
72         t[0] = t[0] + 2;
73         b = Multiply(b, t);
74         b.resize(m);
75     }
76     return b;
77 }
78 static Poly Derivative(const Poly &a)
79 {
80     Poly b;
81     for ( int i = 1; i < (int)a.size(); i++ )
82         b.push_back(a[i] * i);
83     return b;
84 }
85 static Poly Integral(const Poly &a)
86 {
87     Poly b(a.size() + 1);
88     vector<Z> Inv(a.size() + 1);
89     if ( !a.empty() )
90         Inv[1] = 1;
91     for ( int i = 2; i <= (int)a.size(); i++ )
92         Inv[i] = Z(mod - mod / i) * Inv[mod % i];
93     for ( int i = 0; i < (int)a.size(); i++ )
94         b[i + 1] = a[i] * Inv[i + 1];
95     return b;
96 }
97 static Poly Log(const Poly &a, int n)
98 {
99     assert(n > 0 && !a.empty() && a[0].v == 1);
100     auto b = Multiply(Derivative(a), Inverse(a, n));
101     b.resize(n - 1);
102     return Integral(b);
103 }
104 static Poly Exp(const Poly &a, int n)
105 {
106     assert(n > 0 && (a.empty() || a[0].v == 0));
107     Poly b{1};
108     while ( (int)b.size() < n )
109     {
110         int m = min(n, 2 * (int)b.size());
111         auto t = Log(b, m);
112         for ( int i = 0; i < m; i++ )
113             t[i] = (i < (int)a.size() ? a[i] : Z(0)) - t[i];
114         t[0] = t[0] + 1;
115         b = Multiply(b, t);
116         b.resize(m);
117     }
118     return b;
119 }
120 static void Fwt(Poly &a, char op, bool invert = false)
121 {
122     int n = (int)a.size();
123     assert(n > 0 && (n & (n - 1)) == 0);
124     const Z inv2 = (mod + 1) / 2;
125     for ( int len = 1; len < n; len *= 2 )
126         for ( int i = 0; i < n; i += len * 2 )
127             for ( int j = 0; j < len; j++ )
128                 {
129                     Z &u = a[i + j], &v = a[i + j + len], x = u, y = v;
130                     if ( op == '^' )

```

```

131         {
132             u = x + y;
133             v = x - y;
134             if ( invert )
135             {
136                 u = u * inv2;
137                 v = v * inv2;
138             }
139         }
140         else if ( op == '|' )
141             v = invert ? y - x : y + x;
142         else
143         {
144             assert(op == '&');
145             u = invert ? x - y : x + y;
146         }
147     }
148 }
149 // c: s[n]=sum c[i-1]*s[n-i], i=1..L.
150 static Poly Berlekamp_Massey(const Poly &s)
151 {
152     Poly c{1}, b{1};
153     int l = 0, m = 1;
154     Z last = 1;
155     for ( int n = 0; n < (int)s.size(); n++ )
156     {
157         Z d = s[n];
158         for ( int i = 1; i <= l; i++ )
159             d = d + c[i] * s[n - i];
160         if ( !d.v )
161         {
162             ++m;
163             continue;
164         }
165         auto old = c;
166         Z coef = d / last;
167         if ( c.size() < b.size() + m )
168             c.resize(b.size() + m);
169         for ( int j = 0; j < (int)b.size(); j++ )
170             c[j + m] = c[j + m] - coef * b[j];
171         if ( 2 * l <= n )
172         {
173             l = n + 1 - l;
174             b = old;
175             last = d;
176             m = 1;
177         }
178         else
179             ++m;
180     }
181     c.erase(c.begin());
182     for ( auto &x : c )
183         x = Z(0) - x;
184     return c;
185 }
186 static Z Recurrence(Poly init, const Poly &c, unsigned long long n)
187 {
188     int k = (int)c.size();
189     if ( n < init.size() )
190         return init[n];
191     if ( !k )
192         return 0;
193     assert((int)init.size() >= k);
194     auto combine = [&](const Poly &a, const Poly &b)
195     {
196         Poly t(2 * k - 1);
197         for ( int i = 0; i < k; i++ )
198             for ( int j = 0; j < k; j++ )
199                 t[i + j] = t[i + j] + a[i] * b[j];
200         for ( int i = 2 * k - 2; i >= k; i-- )
201             for ( int j = 1; j <= k; j++ )

```

```
202         t[i - j] = t[i - j] + t[i] * c[j - 1];
203     t.resize(k);
204     return t;
205 };
206 Poly a(k), r(k);
207 r[0] = 1;
208 if ( k == 1 )
209     a[0] = c[0];
210 else
211     a[1] = 1;
212 for ( ; n; n >>= 1, a = combine(a, a) )
213     if ( n & 1 )
214         r = combine(r, a);
215 Z ans = 0;
216 for ( int i = 0; i < k; i++ )
217     ans = ans + r[i] * init[i];
218 return ans;
219 }
220 };
```

第二十章 代数与数论进阶

20.1 模高斯消元、行列式、矩阵树定理

算法约束见紧凑版本第 22 页。本节代码独立可抄；采用下列实际接口名。

```
1 template <int mod> struct Linear_Algebra
2 {
3     using Z = Mod_Int<mod>;
4     using Matrix = vector<vector<Z>>;
5     struct Solution
6     {
7         bool consistent;
8         int rank;
9         vector<Z> particular;
10        Matrix kernel;
11    };
12    // Augmented m x (n+1), prime modulus. Empty system needs explicit n.
13    static Solution Solve(Matrix a, int n)
14    {
15        int m = (int)a.size(), row = 0;
16        vector<int> where(n, -1);
17        for ( const auto &v : a )
18            assert((int)v.size() == n + 1);
19        for ( int col = 0; col < n && row < m; col++ )
20        {
21            int p = row;
22            while ( p < m && !a[p][col].v )
23                ++p;
24            if ( p == m )
25                continue;
26            swap(a[p], a[row]);
27            Z Inv = a[row][col].Inv();
28            for ( int j = col; j <= n; j++ )
29                a[row][j] = a[row][j] * Inv;
30            for ( int i = 0; i < m; i++ )
31                if ( i != row && a[i][col].v )
32                {
33                    Z f = a[i][col];
34                    for ( int j = col; j <= n; j++ )
35                        a[i][j] = a[i][j] - f * a[row][j];
36                }
37            where[col] = row++;
38        }
39        for ( int i = row; i < m; i++ )
40            if ( a[i][n].v )
41                return {false, row, {}, {}};
42        Solution ans{true, row, vector<Z>(n), {}};
43        for ( int j = 0; j < n; j++ )
44            if ( where[j] != -1 )
45                ans.particular[j] = a[where[j]][n];
46        for ( int j = 0; j < n; j++ )
47            if ( where[j] == -1 )
48            {
49                vector<Z> v(n);
50                v[j] = 1;
51                for ( int k = 0; k < n; k++ )
52                    if ( where[k] != -1 )
53                        v[k] = Z(0) - a[where[k]][j];
54                ans.kernel.push_back(v);
55            }
56        return ans;
57    }
58    static Z Determinant(Matrix a)
59    {
```

```

60     int n = (int)a.size();
61     Z ans = 1;
62     for ( int i = 0; i < n; i++ )
63     {
64         assert((int)a[i].size() == n);
65         int p = i;
66         while ( p < n && !a[p][i].v )
67             ++p;
68         if ( p == n )
69             return 0;
70         if ( p != i )
71         {
72             swap(a[p], a[i]);
73             ans = Z(0) - ans;
74         }
75         ans = ans * a[i][i];
76         Z Inv = a[i][i].Inv();
77         for ( int j = i + 1; j < n; j++ )
78         {
79             Z f = a[j][i] * Inv;
80             for ( int k = i; k < n; k++ )
81                 a[j][k] = a[j][k] - f * a[i][k];
82         }
83     }
84     return ans;
85 }
86 static Matrix Multiply(const Matrix &a, const Matrix &b)
87 {
88     assert(!a.empty() && !b.empty());
89     int n = a.size(), m = b[0].size(), k = b.size();
90     assert((int)a[0].size() == k);
91     Matrix c(n, vector<Z>(m));
92     for ( int i = 0; i < n; i++ )
93         for ( int t = 0; t < k; t++ )
94             for ( int j = 0; j < m; j++ )
95                 c[i][j] = c[i][j] + a[i][t] * b[t][j];
96     return c;
97 }
98 static Matrix Power(Matrix a, unsigned long long e)
99 {
100     int n = a.size();
101     assert(n > 0 && (int)a[0].size() == n);
102     Matrix r(n, vector<Z>(n));
103     for ( int i = 0; i < n; i++ )
104         r[i][i] = 1;
105     for ( ; e >= 1, a = Multiply(a, a) )
106         if ( e & 1 )
107             r = Multiply(r, a);
108     return r;
109 }
110 // Undirected multigraph, vertices 0..n-1; loops ignored.
111 static Z Spanning_Trees(int n, const vector<pair<int, int>> &edges)
112 {
113     assert(n > 0);
114     Matrix lap(n, vector<Z>(n));
115     for ( auto [u, v] : edges )
116         if ( u != v )
117         {
118             lap[u][u] = lap[u][u] + 1;
119             lap[v][v] = lap[v][v] + 1;
120             lap[u][v] = lap[u][v] - 1;
121             lap[v][u] = lap[v][u] - 1;
122         }
123     lap.pop_back();
124     for ( auto &row : lap )
125         row.pop_back();
126     return Determinant(lap);
127 }
128 };

```

20.2 杜教筛

算法约束见紧凑版本第 23 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Du_Jiao
2 {
3     using ll = long long;
4     using I = __int128_t;
5     int limit;
6     vector<ll> pmu, pphi;
7     unordered_map<ll, ll> mmu;
8     unordered_map<ll, I> mphi;
9     Du_Jiao(int limit) : limit(limit), pmu(limit + 1), pphi(limit + 1)
10    {
11        assert(limit >= 1);
12        Linear_Sieve s(limit);
13        for ( int i = 1; i <= limit; i++ )
14        {
15            pmu[i] = pmu[i - 1] + s.mu[i];
16            pphi[i] = pphi[i - 1] + s.phi[i];
17        }
18    }
19    ll Mertens(ll n)
20    {
21        if ( n <= limit )
22            return pmu[n];
23        if ( mmu.count(n) )
24            return mmu[n];
25        ll ans = 1;
26        for ( ll l = 2, r; l <= n; l = r + 1 )
27        {
28            r = n / (n / l);
29            ans -= (r - l + 1) * Mertens(n / l);
30        }
31        return mmu[n] = ans;
32    }
33    I Totient_Sum(ll n)
34    {
35        if ( n <= limit )
36            return pphi[n];
37        if ( mphi.count(n) )
38            return mphi[n];
39        I ans = I(n) * (n + 1) / 2;
40        for ( ll l = 2, r; l <= n; l = r + 1 )
41        {
42            r = n / (n / l);
43            ans -= I(r - l + 1) * Totient_Sum(n / l);
44        }
45        return mphi[n] = ans;
46    }
47 };

```

20.3 扩展 BSGS

算法约束见紧凑版本第 24 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Discrete_Log
2 {
3     using ll = long long;
4     // 1<=m<=1e12; O(sqrt(m)) time/storage. Smallest x>=0, or -1.
5     static ll Solve(ll a, ll b, ll m)
6     {
7         assert(m >= 1 && m <= 1000000000000LL);
8         a = (a % m + m) % m;
9         b = (b % m + m) % m;
10        if ( m == 1 )
11            return 0;
12        ll offset = 0, k = 1;
13        for ( ll g; (g = gcd(a, m)) > 1; )
14        {
15            if ( b == k )
16                return offset;
17            if ( b % g )
18                return -1;
19            b /= g;
20            m /= g;
21            k = (__int128)k * (a / g) % m;
22            ++offset;
23        }
24        ll target = (__int128)b * Number_Theory::Inverse(k, m) % m;
25        ll step = sqrtl(m) + 1;
26        unordered_map<ll, ll> baby;
27        ll cur = 1 % m;
28        for ( ll j = 0; j < step; j++ )
29        {
30            if ( !baby.count(cur) )
31                baby[cur] = j;
32            cur = (__int128)cur * a % m;
33        }
34        ll Inv = Number_Theory::Inverse(Number_Theory::Power(a, step, m), m);
35        cur = target;
36        for ( ll i = 0; i <= step; i++ )
37        {
38            auto it = baby.find(cur);
39            if ( it != baby.end() )
40                return offset + i * step + it->second;
41            cur = (__int128)cur * Inv % m;
42        }
43        return -1;
44    }
45 };

```

第二十一章 计算几何

21.1 整数精确二维几何

算法约束见紧凑版本第 25 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Integer_Geometry
2 {
3     using ll = long long;
4     using I = __int128_t;
5     // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
6     struct Point
7     {
8         ll x = 0, y = 0;
9         bool operator<(Point b) const
10         {
11             return tie(x, y) < tie(b.x, b.y);
12         }
13         bool operator==(Point b) const
14         {
15             return x == b.x && y == b.y;
16         }
17     };
18     static I Cross(Point a, Point b, Point c)
19     {
20         return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
21     }
22     static I Dot(Point a, Point b, Point c)
23     {
24         return (I(b.x) - a.x) * (I(c.x) - a.x) + (I(b.y) - a.y) * (I(c.y) - a.y);
25     }
26     static I Distance_Squared(Point a, Point b)
27     {
28         return Dot(a, b, b);
29     }
30     static int Sign(I x)
31     {
32         return (x > 0) - (x < 0);
33     }
34     static bool On_Segment(Point p, Point a, Point b)
35     {
36         return Cross(a, b, p) == 0 && Dot(p, a, b) <= 0;
37     }
38     static bool Intersect(Point a, Point b, Point c, Point d)
39     {
40         int x = Sign(Cross(a, b, c)), y = Sign(Cross(a, b, d));
41         int Z_Function = Sign(Cross(c, d, a)), w = Sign(Cross(c, d, b));
42         if ( !x && On_Segment(c, a, b) )
43             return true;
44         if ( !y && On_Segment(d, a, b) )
45             return true;
46         if ( !Z_Function && On_Segment(a, c, d) )
47             return true;
48         if ( !w && On_Segment(b, c, d) )
49             return true;
50         return x * y < 0 && Z_Function * w < 0;
51     }
52     static vector<Point> Convex_Hull(vector<Point> p)
53     {
54         sort(p.begin(), p.end());
55         p.erase(unique(p.begin(), p.end()), p.end());
56         int n = (int)p.size();
57         if ( n < 3 )
58             return p;
59         vector<Point> h;
```

```

60     for ( Point v : p )
61     {
62         while ( h.size() > 1 && Cross(h[h.size() - 2], h.back(), v) <= 0 )
63             h.pop_back();
64         h.push_back(v);
65     }
66     int low = (int)h.size();
67     for ( int i = n - 2; i >= 0; i-- )
68     {
69         while ( (int)h.size() > low && Cross(h[h.size() - 2], h.back(), p[i]) <= 0 )
70             h.pop_back();
71         h.push_back(p[i]);
72     }
73     h.pop_back();
74     return h; // CCW, no repeated endpoint, no collinear interior points.
75 }
76 static I Area_Twice(const vector<Point> &p)
77 {
78     I ans = 0;
79     int n = (int)p.size();
80     for ( int i = 0; i < n; i++ )
81         ans += I(p[i].x) * p[(i + 1) % n].y - I(p[i].y) * p[(i + 1) % n].x;
82     return ans;
83 }
84 // 0 outside, 1 boundary, 2 inside. Simple polygon; either orientation.
85 static int Contains(const vector<Point> &p, Point q)
86 {
87     int winding = 0, n = (int)p.size();
88     for ( int i = 0; i < n; i++ )
89     {
90         Point a = p[i], b = p[(i + 1) % n];
91         if ( On_Segment(q, a, b) )
92             return 1;
93         I c = Cross(a, b, q);
94         if ( a.y <= q.y && b.y > q.y && c > 0 )
95             ++winding;
96         if ( a.y > q.y && b.y <= q.y && c < 0 )
97             --winding;
98     }
99     return winding ? 2 : 0;
100 }
101 // Input must be the strict CCW hull above. O(log n).
102 static int Convex_Contains(const vector<Point> &p, Point q)
103 {
104     int n = (int)p.size();
105     if ( n < 3 )
106         return Contains(p, q);
107     I a = Cross(p[0], p[1], q), b = Cross(p[0], p[n - 1], q);
108     if ( a < 0 || b > 0 )
109         return 0;
110     if ( !a )
111         return On_Segment(q, p[0], p[1]) ? 1 : 0;
112     if ( !b )
113         return On_Segment(q, p[0], p[n - 1]) ? 1 : 0;
114     int l = 1, r = n - 1;
115     while ( l + 1 < r )
116     {
117         int m = (l + r) / 2;
118         if ( Cross(p[0], p[m], q) >= 0 )
119             l = m;
120         else
121             r = m;
122     }
123     I c = Cross(p[l], p[r], q);
124     return c < 0 ? 0 : c == 0 ? 1 : 2;
125 }
126 static I Diameter_Squared(const vector<Point> &p)
127 {
128     int n = (int)p.size();
129     if ( n < 2 )
130         return 0;

```

```

131     if ( n == 2 )
132         return Distance_Squared(p[0], p[1]);
133     I ans = 0;
134     int j = 1;
135     for ( int i = 0; i < n; i++ )
136     {
137         int k = (i + 1) % n;
138         while ( Cross(p[i], p[k], p[(j + 1) % n]) > Cross(p[i], p[k], p[j]) )
139             j = (j + 1) % n;
140         ans =
141             max({ans, Distance_Squared(p[i], p[j]), Distance_Squared(p[k], p[j])});
142     }
143     return ans;
144 }
145 struct PolarLess
146 {
147     static int half(Point p)
148     {
149         return p.y < 0 || (p.y == 0 && p.x < 0);
150     }
151     bool operator()(Point a, Point b) const
152     {
153         if ( half(a) != half(b) )
154             return half(a) < half(b);
155         I c = Cross({0, 0}, a, b);
156         if ( c )
157             return c > 0;
158         return Distance_Squared({0, 0}, a) < Distance_Squared({0, 0}, b);
159     }
160 };
161 };

```

21.2 浮点点线圆构造

算法约束见紧凑版本第 26 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Real_Geometry
2 {
3     using R = long double;
4     // Tolerance is explicit, not used in sorting. Requires finite, well-scaled input.
5     R eps;
6     Real_Geometry(R eps = 1e-12L) : eps(eps)
7     {
8         assert(eps > 0);
9     }
10    struct Point
11    {
12        R x = 0, y = 0;
13        Point operator+(Point b) const
14        {
15            return {x + b.x, y + b.y};
16        }
17        Point operator-(Point b) const
18        {
19            return {x - b.x, y - b.y};
20        }
21        Point operator*(R k) const
22        {
23            return {x * k, y * k};
24        }
25        Point operator/(R k) const
26        {
27            return {x / k, y / k};
28        }
29    };
30    struct Circle
31    {
32        Point o;
33        R r;

```

```

34     };
35     enum class Kind
36     {
37         none,
38         one,
39         two,
40         infinite,
41         degenerate
42     };
43     struct Result
44     {
45         Kind kind;
46         vector<Point> p;
47     };
48     static R Dot(Point a, Point b)
49     {
50         return a.x * b.x + a.y * b.y;
51     }
52     static R Cross(Point a, Point b)
53     {
54         return a.x * b.y - a.y * b.x;
55     }
56     static R norm(Point a)
57     {
58         return hypotl(a.x, a.y);
59     }
60     static Point perp(Point a)
61     {
62         return {-a.y, a.x};
63     }
64     Point Projection(Point p, Point a, Point b) const
65     {
66         Point v = b - a;
67         R d = Dot(v, v);
68         if ( d == 0 )
69             return a;
70         return a + v * (Dot(p - a, v) / d);
71     }
72     R Segment_Distance(Point p, Point a, Point b) const
73     {
74         Point v = b - a;
75         R d = Dot(v, v);
76         if ( d == 0 )
77             return norm(p - a);
78         R t = clamp(Dot(p - a, v) / d, R(0), R(1));
79         return norm(p - (a + v * t));
80     }
81     Result Line_Intersection(Point a, Point b, Point c, Point d) const
82     {
83         Point u = b - a, v = d - c;
84         R nu = norm(u), nv = norm(v);
85         if ( nu == 0 || nv == 0 )
86             return {Kind::degenerate, {}};
87         R det = Cross(u, v);
88         if ( fabsl(det) <= eps * nu * nv )
89             return {fabsl(Cross(c - a, u)) <= eps * nu * max(R(1), norm(c - a))
90                     ? Kind::infinite
91                     : Kind::none,
92                     {}};
93         return {Kind::one, {a + u * (Cross(c - a, v) / det)}};
94     }
95     Result Line_Circle(Point a, Point b, Circle c) const
96     {
97         assert(c.r >= 0);
98         Point v = b - a;
99         R len = norm(v);
100        if ( len == 0 )
101            return {Kind::degenerate, {}};
102        Point h = Projection(c.o, a, b);
103        R d = norm(h - c.o), tol = eps * max({R(1), d, c.r});
104        if ( d > c.r + tol )

```

```

105         return {Kind::none, {}};
106     if ( fabsl(d - c.r) <= tol )
107         return {Kind::one, {h}};
108     R t = sqrtl(max(R(0), (c.r - d) * (c.r + d)));
109     Point w = v * (t / len);
110     return {Kind::two, {h - w, h + w}};
111 }
112 Result Circles(Circle a, Circle b) const
113 {
114     assert(a.r >= 0 && b.r >= 0);
115     Point v = b.o - a.o;
116     R d = norm(v);
117     R tol = eps * max({R(1), d, a.r, b.r});
118     if ( d <= tol )
119     {
120         if ( fabsl(a.r - b.r) > tol )
121             return {Kind::none, {}};
122         if ( a.r == 0 && b.r == 0 )
123             return {Kind::one, {a.o}};
124         return {Kind::infinite, {}};
125     }
126     if ( d > a.r + b.r + tol || d < fabsl(a.r - b.r) - tol )
127         return {Kind::none, {}};
128     R x = (d * d + (a.r - b.r) * (a.r + b.r)) / (2 * d);
129     Point h = a.o + v * (x / d);
130     if ( fabsl(d - (a.r + b.r)) <= tol || fabsl(d - fabsl(a.r - b.r)) <= tol )
131         return {Kind::one, {h}};
132     R y = sqrtl(max(R(0), (a.r - x) * (a.r + x)));
133     Point w = perp(v) * (y / d);
134     return {Kind::two, {h - w, h + w}};
135 }
136 R Overlap(Circle a, Circle b) const
137 {
138     R d = norm(a.o - b.o), pi = acosl(-1.L);
139     if ( d >= a.r + b.r )
140         return 0;
141     if ( d <= fabsl(a.r - b.r) )
142         return pi * min(a.r, b.r) * min(a.r, b.r);
143     R x =
144         acosl(clamp((d * d + a.r * a.r - b.r * b.r) / (2 * d * a.r), R(-1), R(1)));
145     R y =
146         acosl(clamp((d * d + b.r * b.r - a.r * a.r) / (2 * d * b.r), R(-1), R(1)));
147     return a.r * a.r * (x - sinl(2 * x) / 2) + b.r * b.r * (y - sinl(2 * y) / 2);
148 }
149 };

```

第二十二章 优化与可持久化

22.1 Max-plus 矩阵幂

算法约束见紧凑版本第 28 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Max_Plus_Matrix
2 {
3     using I = __int128_t;
4     static constexpr I neg = -(I(1) << 120);
5     int n;
6     vector<vector<I>> a;
7     Max_Plus_Matrix(int n, bool identity = false) : n(n), a(n, vector<I>(n, neg))
8     {
9         if ( identity )
10             for ( int i = 0; i < n; i++ )
11                 a[i][i] = 0;
12     }
13     Max_Plus_Matrix operator*(const Max_Plus_Matrix &b) const
14     {
15         assert(n == b.n);
16         Max_Plus_Matrix c(n);
17         for ( int i = 0; i < n; i++ )
18             for ( int k = 0; k < n; k++ )
19                 if ( a[i][k] != neg )
20                     for ( int j = 0; j < n; j++ )
21                         if ( b.a[k][j] != neg )
22                             c.a[i][j] = max(c.a[i][j], a[i][k] + b.a[k][j]);
23         return c;
24     }
25     Max_Plus_Matrix Power(unsigned long long e) const
26     {
27         Max_Plus_Matrix b = *this, r(n, true);
28         for ( ; e >= 1; b = b * b )
29             if ( e & 1 )
30                 r = r * b;
31         return r;
32     }
33 };
```

22.2 离散李超树与最优直线编号

算法约束见紧凑版本第 29 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Li_Chao_Tree
2 {
3     using ll = long long;
4     using I = __int128_t;
5     struct Line
6     {
7         ll k = 0, b = 0;
8         int id = -1;
9         I value(ll x) const
10         {
11             return I(k) * x + b;
12         }
13     };
14     struct Node
15     {
16         Line line;
17         int left = -1, right = -1;
18     };
19     vector<ll> xs;
```

```

20 vector<Node> a{Node{}};
21 Li_Chao_Tree(vector<ll> x) : xs(x)
22 {
23     sort(xs.begin(), xs.end());
24     xs.erase(unique(xs.begin(), xs.end()), xs.end());
25     assert(!xs.empty());
26 }
27 bool better(Line u, Line v, ll x) const
28 {
29     return v.id == -1 ||
30         (u.id != -1 && pair{u.value(x), u.id} < pair{v.value(x), v.id});
31 }
32 void Insert(Line line)
33 {
34     assert(line.id >= 0);
35     Insert(0, 0, (int)xs.size() - 1, line);
36 }
37 void Insert(int p, int l, int r, Line line)
38 {
39     int m = (l + r) / 2;
40     if ( better(line, a[p].line, xs[m]) )
41         swap(line, a[p].line);
42     if ( l == r || line.id == -1 )
43         return;
44     bool left = better(line, a[p].line, xs[l]);
45     bool right = better(line, a[p].line, xs[r]);
46     if ( !left && !right )
47         return;
48     int next = left ? a[p].left : a[p].right;
49     if ( next == -1 )
50     {
51         next = (int)a.size();
52         a.push_back(Node{});
53         if ( left )
54             a[p].left = next;
55         else
56             a[p].right = next;
57     }
58     if ( left )
59         Insert(next, l, m, line);
60     else
61         Insert(next, m + 1, r, line);
62 }
63 pair<I, int> Query(ll x) const
64 {
65     int idx = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
66     assert(idx < (int)xs.size() && xs[idx] == x);
67     int p = 0, l = 0, r = (int)xs.size() - 1;
68     Line best;
69     while ( p != -1 )
70     {
71         if ( better(a[p].line, best, x) )
72             best = a[p].line;
73         if ( l == r )
74             break;
75         int m = (l + r) / 2;
76         if ( idx <= m )
77         {
78             p = a[p].left;
79             r = m;
80         }
81         else
82         {
83             p = a[p].right;
84             l = m + 1;
85         }
86     }
87     assert(best.id != -1);
88     return {best.value(x), best.id};
89 }
90 };

```

22.3 主席树：静态区间第 k 小

算法约束见紧凑版本第 30 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Persistent_Kth
2 {
3     struct Node
4     {
5         int l = 0, r = 0, sum = 0;
6     };
7     vector<Node> t{Node{}};
8     vector<int> root{0};
9     vector<long long> vals;
10    Persistent_Kth(const vector<long long> &a) : vals(a)
11    {
12        sort(vals.begin(), vals.end());
13        vals.erase(unique(vals.begin(), vals.end()), vals.end());
14        for ( long long x : a )
15        {
16            int p = lower_bound(vals.begin(), vals.end(), x) - vals.begin();
17            root.push_back(Insert(root.back(), 0, (int)vals.size() - 1, p));
18        }
19    }
20    int Insert(int old, int l, int r, int x)
21    {
22        int p = t.size();
23        Node copy = t[old];
24        t.push_back(copy);
25        ++t[p].sum;
26        if ( l < r )
27        {
28            int m = (l + r) / 2;
29            if ( x <= m )
30            {
31                int v = Insert(t[old].l, l, m, x);
32                t[p].l = v;
33            }
34            else
35            {
36                int v = Insert(t[old].r, m + 1, r, x);
37                t[p].r = v;
38            }
39        }
40        return p;
41    }
42    long long Kth(int l, int r, int k) const
43    {
44        assert(1 <= l && l <= r && r < (int)root.size() && 1 <= k && k <= r - l + 1);
45        int u = root[l - 1], v = root[r], L = 0, R = (int)vals.size() - 1;
46        while ( L < R )
47        {
48            int m = (L + R) / 2, c = t[t[v].l].sum - t[t[u].l].sum;
49            if ( k <= c )
50            {
51                u = t[u].l;
52                v = t[v].l;
53                R = m;
54            }
55            else
56            {
57                k -= c;
58                u = t[u].r;
59                v = t[v].r;
60                L = m + 1;
61            }
62        }
63        return vals[L];
64    }
65 };

```

第二十三章 图论进阶

23.1 矩形二分图最小权匹配

算法约束见紧凑版本第 31 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Kuhn_Munkres
2 {
3     using I = __int128_t;
4     // Minimum cost injection from n rows to m columns, n<=m. Zero-based matrix.
5     static pair<I, vector<int>> Solve(const vector<vector<long long>> &cost)
6     {
7         int n = cost.size();
8         if ( !n )
9             return {0, {}};
10        int m = cost[0].size();
11        assert(n <= m);
12        vector<I> u(n + 1), v(m + 1);
13        vector<int> p(m + 1), way(m + 1);
14        const I inf = I(1) << 120;
15        for ( int i = 1; i <= n; i++ )
16        {
17            assert((int)cost[i - 1].size() == m);
18            p[0] = i;
19            int j0 = 0;
20            vector<I> dis(m + 1, inf);
21            vector<bool> Used(m + 1);
22            do
23            {
24                Used[j0] = true;
25                int i0 = p[j0], j1 = 0;
26                I delta = inf;
27                for ( int j = 1; j <= m; j++ )
28                    if ( !Used[j] )
29                    {
30                        I cur = I(cost[i0 - 1][j - 1]) - u[i0] - v[j];
31                        if ( cur < dis[j] )
32                        {
33                            dis[j] = cur;
34                            way[j] = j0;
35                        }
36                        if ( dis[j] < delta )
37                        {
38                            delta = dis[j];
39                            j1 = j;
40                        }
41                    }
42                for ( int j = 0; j <= m; j++ )
43                    if ( Used[j] )
44                    {
45                        u[p[j]] += delta;
46                        v[j] -= delta;
47                    }
48                else
49                    dis[j] -= delta;
50                j0 = j1;
51            } while ( p[j0] );
52            do
53            {
54                int j1 = way[j0];
55                p[j0] = p[j1];
56                j0 = j1;
57            } while ( j0 );
58        }
59        vector<int> ans(n);
```

```

60     for ( int j = 1; j <= m; j++ )
61         if ( p[j] )
62             ans[p[j] - 1] = j - 1;
63     return {-v[0], ans};
64 }
65 };

```

23.2 有向最小树形图

算法约束见紧凑版本第 32 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Directed_MST
2 {
3     using I = __int128_t;
4     struct Edge
5     {
6         int u, v;
7         I w;
8     };
9     // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
10    static optional<I> Solve(int n, int root, vector<Edge> e)
11    {
12        const I inf = I(1) << 120;
13        I ans = 0;
14        while ( true )
15        {
16            vector<I> in(n, inf);
17            vector<int> pre(n, -1), id(n, -1), vis(n, -1);
18            for ( auto a : e )
19                if ( a.u != a.v && a.w < in[a.v] )
20                {
21                    in[a.v] = a.w;
22                    pre[a.v] = a.u;
23                }
24            in[root] = 0;
25            for ( I x : in )
26            {
27                if ( x == inf )
28                    return nullopt;
29                ans += x;
30            }
31            int cnt = 0;
32            for ( int i = 0; i < n; i++ )
33            {
34                int u = i;
35                while ( vis[u] != i && id[u] == -1 && u != root )
36                {
37                    vis[u] = i;
38                    u = pre[u];
39                }
40                if ( u != root && id[u] == -1 )
41                {
42                    id[u] = cnt;
43                    for ( int v = pre[u]; v != u; v = pre[v] )
44                        id[v] = cnt;
45                    ++cnt;
46                }
47            }
48            if ( !cnt )
49                return ans;
50            for ( int i = 0; i < n; i++ )
51                if ( id[i] == -1 )
52                    id[i] = cnt++;
53            for ( auto &a : e )
54            {
55                int v = a.v;
56                a.u = id[a.u];
57                a.v = id[a.v];
58                if ( a.u != a.v )

```

```

59         a.w -= in[v];
60     }
61     root = id[root];
62     n = cnt;
63 }
64 }
65 };

```

23.3 无向全局最小割

算法约束见紧凑版本第 33 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Global_Min_Cut
2 {
3     using I = __int128_t;
4     // Symmetric nonnegative adjacency matrix; returns weight and one cut side.
5     static pair<I, vector<int>> Solve(vector<vector<I>> w)
6     {
7         int n = w.size();
8         assert(n >= 2);
9         vector<int> active(n);
10        iota(active.begin(), active.end(), 0);
11        vector<vector<int>> group(n);
12        for ( int i = 0; i < n; i++ )
13            group[i] = {i};
14        I best = I(1) << 120;
15        vector<int> side;
16        while ( active.size() > 1 )
17        {
18            vector<I> dis(n);
19            vector<bool> Used(n);
20            int prev = -1;
21            for ( int step = 0; step < (int)active.size(); step++ )
22            {
23                int sel = -1;
24                for ( int v : active )
25                    if ( !Used[v] && (sel == -1 || dis[v] > dis[sel]) )
26                        sel = v;
27                if ( step + 1 == (int)active.size() )
28                {
29                    if ( dis[sel] < best )
30                    {
31                        best = dis[sel];
32                        side = group[sel];
33                    }
34                    for ( int v : active )
35                        if ( v != prev && v != sel )
36                            w[prev][v] = w[v][prev] = w[prev][v] + w[sel][v];
37                    group[prev].insert(group[prev].end(), group[sel].begin(),
38                                     group[sel].end());
39                    active.erase(find(active.begin(), active.end(), sel));
40                    break;
41                }
42                Used[sel] = true;
43                prev = sel;
44                for ( int v : active )
45                    if ( !Used[v] )
46                        dis[v] += w[sel][v];
47            }
48        }
49        return {best, side};
50    }
51 };

```

第二十四章 精确几何进阶

24.1 整数最近点对与 Minkowski 和

算法约束见紧凑版本第 34 页。本节代码独立可抄；采用下列实际接口名。

```
1 struct Geometry_Extra
2 {
3     using G = Integer_Geometry;
4     using P = G::Point;
5     using I = G::I;
6     // No pair exists for n<2. Exact squared distance, O(n log n).
7     static optional<I> Closest_Pair(vector<P> p)
8     {
9         int n = p.size();
10        if ( n < 2 )
11            return nullopt;
12        sort(p.begin(), p.end());
13        for ( int i = 1; i < n; i++ )
14            if ( p[i] == p[i - 1] )
15                return I(0);
16        vector<P> tmp(n);
17        auto by_y = [](P a, P b) { return tie(a.y, a.x) < tie(b.y, b.x); };
18        function<I(int, int)> Solve = [&](int l, int r) -> I
19        {
20            if ( r - l <= 3 )
21            {
22                I best = I(1) << 120;
23                for ( int i = l; i < r; i++ )
24                    for ( int j = i + 1; j < r; j++ )
25                        best = min(best, G::Distance_Squared(p[i], p[j]));
26                sort(p.begin() + l, p.begin() + r, by_y);
27                return best;
28            }
29            int m = (l + r) / 2;
30            long long x = p[m].x;
31            I best = min(Solve(l, m), Solve(m, r));
32            merge(p.begin() + l, p.begin() + m, p.begin() + m, p.begin() + r,
33                tmp.begin() + l, by_y);
34            copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
35            vector<P> strip;
36            for ( int i = l; i < r; i++ )
37                if ( (I(p[i].x) - x) * (I(p[i].x) - x) < best )
38                {
39                    for ( int j = (int)strip.size() - 1; j >= 0; j-- )
40                    {
41                        I dy = I(p[i].y) - strip[j].y;
42                        if ( dy * dy >= best )
43                            break;
44                        best = min(best, G::Distance_Squared(p[i], strip[j]));
45                    }
46                    strip.push_back(p[i]);
47                }
48            return best;
49        };
50        return Solve(0, n);
51    }
52    // Inputs strict CCW hulls. Coordinate sums must obey IntegerGeometry's bound.
53    static vector<P> Minkowski(vector<P> a, vector<P> b)
54    {
55        if ( a.empty() || b.empty() )
56            return {};
57        auto plus = [](P u, P v) { return P{u.x + v.x, u.y + v.y}; };
58        if ( a.size() < 3 || b.size() < 3 )
59        {
```

```

60     vector<P> p;
61     for ( P u : a )
62         for ( P v : b )
63             p.push_back(plus(u, v));
64     return G::Convex_Hull(p);
65 }
66 auto start = [] (vector<P> &p)
67 {
68     auto it = min_element(p.begin(), p.end(), [](P u, P v)
69         { return tie(u.y, u.x) < tie(v.y, v.x); });
70     rotate(p.begin(), it, p.end());
71 };
72 start(a);
73 start(b);
74 int n = a.size(), m = b.size(), i = 0, j = 0;
75 vector<P> result;
76 while ( i < n || j < m )
77 {
78     result.push_back(plus(a[i % n], b[j % m]));
79     if ( i == n )
80     {
81         ++j;
82         continue;
83     }
84     if ( j == m )
85     {
86         ++i;
87         continue;
88     }
89     P u{a[(i + 1) % n].x - a[i].x, a[(i + 1) % n].y - a[i].y};
90     P v{b[(j + 1) % m].x - b[j].x, b[(j + 1) % m].y - b[j].y};
91     I c = G::Cross({0, 0}, u, v);
92     if ( c >= 0 )
93         ++i;
94     if ( c <= 0 )
95         ++j;
96 }
97 return result;
98 }
99 };

```

24.2 整数三维共线、共面与方向

算法约束见紧凑版本第 35 页。本节代码独立可抄；采用下列实际接口名。

```

1 struct Integer_Geometry_3D
2 {
3     using I = __int128_t;
4     // |coordinates|<=1e9: triple products fit signed int128.
5     struct Point
6     {
7         long long x, y, Z_Function;
8     };
9     struct Vector
10    {
11        I x, y, Z_Function;
12    };
13    static Vector diff(Point a, Point b)
14    {
15        return {I(a.x) - b.x, I(a.y) - b.y, I(a.Z_Function) - b.Z_Function};
16    }
17    static Vector Cross(Vector a, Vector b)
18    {
19        return {a.y * b.Z_Function - a.Z_Function * b.y,
20                a.Z_Function * b.x - a.x * b.Z_Function, a.x * b.y - a.y * b.x};
21    }
22    static I Dot(Vector a, Vector b)
23    {
24        return a.x * b.x + a.y * b.y + a.Z_Function * b.Z_Function;
25    }
26    static I orient(Point a, Point b, Point c, Point d)
27    {
28        return Dot(Cross(diff(b, a), diff(c, a)), diff(d, a));
29    }
30    static bool collinear(Point a, Point b, Point c)
31    {
32        auto v = Cross(diff(b, a), diff(c, a));
33        return !v.x && !v.y && !v.Z_Function;
34    }
35    static bool On_Segment(Point p, Point a, Point b)
36    {
37        return collinear(p, a, b) && Dot(diff(a, p), diff(b, p)) <= 0;
38    }
39 };

```

第三部分

数学速查与维护

第二十五章 数学速查：条件先于公式

25.1 整除、卷积与筛法

对正整数定义狄利克雷卷积

$$(f * g)(n) = \sum_{d|n} f(d)g(n/d).$$

单位元为 $\varepsilon(n) = [n = 1]$, 常函数 $1(n) = 1$, 恒等函数 $\text{id}(n) = n$ 。关键恒等式是 $\mu * 1 = \varepsilon$ 、 $\varphi * 1 = \text{id}$ 。若 $F(n) = \sum_{d|n} f(d)$, 则 $f(n) = \sum_{d|n} \mu(d)F(n/d)$ 。

$$\sum_{i=1}^a \sum_{j=1}^b [\gcd(i, j) = 1] = \sum_{d=1}^{\min(a, b)} \mu(d) \lfloor a/d \rfloor \lfloor b/d \rfloor.$$

整除分块：当左端点为 l 、商为 $q = \lfloor n/l \rfloor$, 右端点为 $r = \lfloor n/q \rfloor$ 。同时包含多个商时，取各自右端点的最小值。杜教筛由卷积恒等式获得

$$M(n) = 1 - \sum_{l=2}^n M(\lfloor n/l \rfloor), \quad \Phi(n) = \frac{n(n+1)}{2} - \sum_{l=2}^n \Phi(\lfloor n/l \rfloor),$$

求和按等商区间合并；第二式求和中没有额外乘以 l 。

25.2 模运算的适用条件

模逆元存在当且仅当 $\gcd(a, m) = 1$ 。费马逆元 a^{p-2} 仅适用于素数模数及 $a \not\equiv 0$ 。对正模数 m, n , 同余式 $x \equiv a \pmod{m}$ 、 $x \equiv b \pmod{n}$ 可合并当且仅当 $\gcd(m, n) \mid (b - a)$ ；新模数为 $\text{lcm}(m, n)$ 。

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p} \quad (p \text{ 为素数, } n_i, k_i \text{ 是 } p \text{ 进制位}).$$

Lucas 不能直接套在合数模数。阶乘与逆阶乘预处理要求上限小于模数；超过模数会出现阶乘为零。扩展欧拉降幂中，非互质情况不能直接将指数对 $\varphi(m)$ 取模；需要判断原指数是否达到足够阈值。大整数指数可以在读入时同时维护余数与是否超过阈值。

25.3 组合计数

容斥： $|\bigcup A_i| = \sum_{\emptyset \neq S} (-1)^{|S|+1} |\bigcap_{i \in S} A_i|$ 。二项式反演：若 $b_n = \sum_{k=0}^n \binom{n}{k} a_k$, 则 $a_n = \sum_{k=0}^n (-1)^{n-k} \binom{n}{k} b_k$ 。

$$\sum_k \binom{a}{k} \binom{b}{n-k} = \binom{a+b}{n}, \quad C_n = \binom{2n}{n} - \binom{2n}{n+1}.$$

差分形式的 Catalan 公式避免直接除以 $n+1$, 在模数下尤其有用，但组合数本身仍须正确求模。第二类 Stirling 数满足

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} + k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\}, \quad S(0, 0) = 1.$$

第一类无符号 Stirling 数把系数 k 换为 $n-1$ 。

球与盒	空盒允许	空盒禁止
不同球，不同盒	k^n	$k!S(n, k)$
相同球，不同盒	$\binom{n+k-1}{k-1}$	$\binom{n-1}{k-1}$
不同球，相同盒	$\sum_{j=0}^k S(n, j)$	$S(n, k)$

上表默认 $k \geq 1$; $n = 0, k = 0$ 等边界应按原问题约定单独处理。

25.4 群作用计数

有限群 G 作用在有限集合 X 上时, 轨道数为

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|.$$

对位置置换的 q 色染色, $|\text{Fix}(g)| = q^{c(g)}$, 其中 $c(g)$ 为循环数。环形项链只商去旋转; 手链还商去反射, 二者群不同, 不能共用分母。模数整除 $|G|$ 时不能使用模逆元, 须先在合适的整数模数下求和再做精确除法。

25.5 生成函数与递推

普通生成函数 $A(x) = \sum a_n x^n$ 的乘法对应普通卷积。指数生成函数 $A(x) = \sum a_n x^n / n!$ 的乘法对应 $c_n = \sum_k \binom{n}{k} a_k b_{n-k}$ 。形式幂级数求逆要求常数项可逆; $\log A$ 取 $A(0) = 1$, $\exp A$ 取 $A(0) = 0$ 。有限域中积分涉及 $1/i$, 必须避免 i 被特征整除。

$$B_{\text{new}} = B(2 - AB) \pmod{x^{2m}}, \quad (\log A)' = A'/A.$$

BM 从样本得到最短线性递推。要对未来项使用, 必须另外证明序列确实具有不超过相应阶数的线性递推; 少量数值吻合不是证明。矩阵转移、有限状态计数常可提供阶数上界。

25.6 线性代数与生成树

域上的线性方程组一致时, 解集为一个特解加零空间, 零空间维数为未知数个数减秩。浮点消元中的秩依赖误差阈值; 模素数消元是精确域运算; 合数模数下不可随意除以非零元。无向图 Laplacian 的对角元为度数, 非对角元为负的边重数。删去同一行列后的行列式为生成树数。自环不参与生成树; 重边需要累加。只有一个顶点时空余子式行列式约定为 1。有向图需要重新明确入度/出度 Laplacian 与根向内/向外树的方向, 不能直接复用无向封装。

25.7 概率、期望与博弈

期望线性性不要求独立: $\mathbb{E}[\sum X_i] = \sum \mathbb{E}[X_i]$ 。方差可加需要协方差为零; 独立是充分条件。非负整数变量 $\mathbb{E}[X] = \sum_{k \geq 1} \Pr(X \geq k)$ 。吸收过程的期望方程 $E_i = 1 + \sum_j p_{ij} E_j$ 需要检查是否几乎必然吸收及期望是否有限; 不能对闭合非吸收分量直接求唯一有限解。有限无环公平组合游戏在正常玩法下

$$\text{SG}(u) = \text{mex}\{\text{SG}(v) : u \rightarrow v\}.$$

独立子游戏的 SG 值异或为零当且仅当先手必败。反常玩法、含环游戏、双方可走步不同的游戏不能直接套这个结论。

25.8 数值算法与几何

排序必须使用严格弱序; 不能以“差小于 ϵ 则相等”定义通用排序比较器。叉积与行列式容易发生抵消; 整数题优先精确整数判定, 浮点构造单独处理误差。对于简单格点多边形 (无孔), Pick 定理 $A = I + B/2 - 1$, 边界格点数 $B = \sum \gcd(|\Delta x|, |\Delta y|)$; 不要重复计入边端点。自适应积分的局部误差估计需要函数足够规则; 奇点、间断与强振荡必须先分段处理。三分搜索要求单峰性, 整数域最后枚举剩余区间; 二分答案要求判定单调性。

来源索引 本章为重新组织的公式速查, 扩展范围依据 <https://oi-wiki.org/math/>。还需补齐 exLucas、二次剩余、原根、Min25、插值、子集卷积、行列式进阶、数值积分等完整实现与验证; 本章公式不能代替这些待完成代码。

第二十六章 来源与覆盖维护

26.1 参考基线

- kuangbin, ACM 常用算法模板, 2018 年版本: <https://kuangbin.github.io/2018/08/01/ACM-template/>。
- WIDA XCPC: <https://github.com/hh2048/XCPC>, 参考快照 484e9e0。
- OI Wiki 数学: <https://oi-wiki.org/math/>。

本项目重新实现的代码与公式速查不能冒用原作者的验证记录。完整覆盖审计应为每个来源条目记录实现文件、替代关系与验证证据, 保留遗漏项直至完成。

26.2 后续覆盖项

尚需完成的范围以覆盖表为准, 包括动态树、平衡树、一般图匹配、最小树形图、上下界流、高阶几何构造、半平面交、三维凸包、数值积分、复杂数论及多项式、搜索与动态规划等。不能用通用模块的存在推断其覆盖了所有原书例题。

算法索引

AC_Automaton, 54
AhoCorasick, 14
Arborescence, 32

Berlekamp-Massey, 87
Binary_Indexed_Tree, 39
Binomial, 19
Bipartite_Matching, 48
BipartiteMatching, 10
Bounded_Circulation, 45
BoundedCirculation, 7
Burnside, 87

Catalan, 86
Combination, 62
CRT, 86

Dijkstra, 8
Dinic, 5
Directed_MST, 80
Dirichlet convolution, 86
Discrete_Log, 70
DiscreteLog, 24
Disjoint_Set, 37
DSU, 2
Du_Jiao, 69
DuJiao, 23

Euler theorem, 86
expectation, 87

Fenwick, 3

generating functions, 87
Geometry_Extra, 82
GeometryExtra, 34
Global_Min_Cut, 81

Heavy_Light_Decomposition, 51
HLD, 12
Hungarian, 31

inclusion-exclusion, 86
Integer_Geometry, 71
Integer_Geometry_3D, 84
IntegerGeometry, 25

IntegerGeometry3D, [35](#)

Kuhn_Munkres, [79](#)

LazySeg, [3](#)

Li_Chao_Tree, [76](#)

LiChao, [29](#)

Linear_Algebra, [67](#)

Linear_Sieve, [61](#)

LinearAlgebra, [22](#)

LinearSieve, [19](#)

Low_Link, [50](#)

Lowlink, [11](#)

Lucas theorem, [86](#)

Matrix Tree theorem, [87](#)

Max_Plus_Matrix, [76](#)

MaxPlusMatrix, [28](#)

Min_Cost_Flow, [43](#)

MinCostFlow, [6](#)

Mobius inversion, [86](#)

Mod_Int, [62](#)

ModInt, [19](#)

Network_Flow, [42](#)

Number_Theory, [58](#)

NumberTheory, [17](#)

numerical stability, [87](#)

Persistent_Kth, [78](#)

PersistentKth, [30](#)

Pick theorem, [87](#)

Pollard_Rho, [60](#)

PollardRho, [18](#)

Polya, [87](#)

Polynomial, [20](#), [63](#)

rank, [87](#)

Real_Geometry, [73](#)

RealGeometry, [26](#)

Rollback_DSU, [38](#)

RollbackDSU, [2](#)

SCC, [9](#)

Segment_Tree, [40](#)

Shortest_Path, [46](#)

Sprague-Grundy, [87](#)

Stirling numbers, [86](#)

StoerWagner, [33](#)

String_Algorithm, [53](#)

StringAlgo, [13](#)

Strong_Component, [46](#)

Suffix_Array, [56](#)

Suffix_Automaton, [56](#)

SuffixArray, [15](#)

SuffixAutomaton, [16](#)

Two_SAT, [48](#)

TwoSAT, [9](#)

Xor_Basis, [41](#)

XorBasis, [4](#)